

Formal Verification of InfluenceDiagrams.jl

Simon Frost

2026-09-08

Contents

InfluenceDiagramsProofs	2
What is formalised	2
Correspondence with SPEC and the Julia API	2
InfluenceDiagramsProofs.Finite.InfluenceDiagram	3
Generators and validity	4
Policies and strategies	5
InfluenceDiagramsProofs.Finite.Instantiate	6
InfluenceDiagramsProofs.Finite.ExpectedUtility	8
Utilities	9
Fixing a decision	10
InfluenceDiagramsProofs.Finite.Information	11
InfluenceDiagramsProofs.Finite.Optimization	12
InfluenceDiagramsProofs.Finite.OptimalInformation	15
Exact probability/utility valuations	16
Structural probability independence	21
Bucket driver and policy reconstruction	25
Initialisation and the returned solution	28
No-forgetting generates the strong schedule	32
Action-independent evidence	34
Nonvacuity: a partially observed two-decision model	41
The exact all-row guard and its positive-input intermediate lemma	43
Probability-expression provenance	46
Exact all-row diagnostic completeness	49
Exact-guard boundary checks	52
Ordered local policies and a quantitative tie-gap contract	55
Roadmap and exact scope of the DVE theorem	56

InfluenceDiagramsProofs

```
import BayesianNetworksProofs.Schema.BayesNet
```

Lean 4 / Mathlib formalisation accompanying InfluenceDiagrams.jl (toolchain leanprover/lean4:v4.30.0, Mathlib tag v4.30.0; ADR 0005). This document is generated from the Lean sources by [mdgen](#): the prose is the module docstrings and the code blocks are the verbatim, machine-checked sources. Every library module is built by `lake build --wfail`; `Audit.lean` prints the axioms of the headline results (only `propext`, `Classical.choice`, `Quot.sound`). The Roadmap has no remaining proof holes.

The project depends by path on `BayesianNetworks.jl/proofs` (Lake package `bayesian_networks_proofs`), which supplies the finite Bayesian-network model (`FinBayesNet`, `kernels`, `Local`, `Normalised`, `Closed`, `TopoOrder`, `joint`, `Propositions 1 and 4`) and the ACSet schema terms; the influence-diagram schema `schInfluenceDiagram` is defined there and re-emitted here by `lake exe emit_schema`.

What is formalised

`InfluenceDiagrams.jl` represents an influence diagram as an ACSet on `SchInfluenceDiagram` (SPEC §24: decisions, information arcs, utilities, decision precedence). A complete strategy turns it into a Bayesian network (`instantiate`, SPEC §28) whose `joint` gives the expected utility (`expected_utility`, SPEC §29–§31); fixing a decision (`fix_decision`, SPEC §41) is an intervention, and adding information arcs cannot lower the optimal expected utility (SPEC §34–§35, §55.7). The algebraic results are over an arbitrary commutative semiring R ; the optimisation and attained-optimum information results use R and nonnegative strategies.

- `Finite/InfluenceDiagram.lean`: decisions, information and utility scopes, validity, policies, complete strategies and fixed decisions.
- `Finite/Instantiate.lean`: Proposition 5, policy substitution, factorisation, locality and normalisation of the instantiated joint.
- `Finite/ExpectedUtility.lean`: Proposition 6, linearity, additive utilities and fixed decisions as hard interventions.
- `Finite/Information.lean`: strategy inclusion under information enlargement, with exactly the same expected utility.
- `Finite/Optimization.lean`: exact convex mixtures of deterministic tables and global deterministic optimality for any finite number of decisions, including limited memory. Signed-weight counterexample.
- `Finite/OptimalInformation.lean`: attainment of the finite optimum, monotonicity of the optimal value and nonnegativity of cost-free information value.
- `Finite/DVE/`: an actual probability/utility bucket driver, strong schedules generated from no-forgetting, local policy reconstruction, exact realized optimality, and positive-mass action-independent evidence. A hidden-state two-decision example checks nonvacuity.
- `Finite/DVE/Guard/`: exact max-minus-min diagnostic completeness on all rows, including zero-outside contexts, proved by probability-expression provenance and normalized smoothing.
- `Finite/OrderedPolicies.lean`: first-state local ties in a supplied finite order, finite information-table reconstruction, and stability under a strict numerical action gap.
- `Roadmap.lean`: remaining literal-Julia refinement boundaries; no unproved declarations.

Correspondence with SPEC and the Julia API

SPEC	Lean	Julia (InfluenceDiagrams.jl)
§26–§27 policies, strategies	<code>Policy</code> , <code>Policy.ofFun</code> , <code>Strategy</code>	<code>DeterministicPolicy</code> , <code>StochasticPolicy</code> , <code>Strategy</code>
§28, §61 Prop 5 — policy instantiation yields a valid BN	<code>instantiate</code> , <code>strategyKernel</code> , <code>closed_instantiate</code> , <code>IDOrder.toTopoOrder</code> , <code>sum_joint_instantiate_eq_one</code> , <code>joint_instantiate</code>	<code>instantiate(id, strategy)</code> $\rightarrow$ <code>BayesNet</code> (<code>validate(closed=true)</code> <code>succeeds</code>)
§29–§31, §61 Prop 6 — $EU(\sigma) = E_{\{P_\sigma\}}[U]$	<code>Utility</code> , <code>totalUtility</code> , <code>expectedUtility</code> , <code>expectedUtility_eq</code> , <code>expectedUtility_eq_sum</code>	<code>expected_utility(id, strategy)</code> (<code>instantiate</code> , enumerate the joint, sum utilities, take the expectation)

SPEC	Lean	Julia (InfluenceDiagrams.jl)
§41 fixed decision	<code>Policy.const, Strategy.fix, strategyKernel_fix_eq_intervene, joint_fix</code>	<code>fix_decision(model, :D ⇒ :a) = do_intervention</code> on the instantiated network
§34–§35, §55.7 information monotonicity	<code>optimalValue_info_mono, optimal_information_value_nonneg</code> , based on attained maxima and <code>expectedUtility_enlarge</code>	cost-free exact expected value of information is nonnegative; no solver correctness asserted
§32 global policy optimisation	<code>exists_deterministic_optimal_all</code> , with the original single-decision theorem as a corollary	finite exhaustive-table optimum, not an implementation proof
§33, §61 Prop 7 — DVE correctness	Actual finite-function bucket solver, generated strong order, realized optimal policies, and exact all-row diagnostic completeness	Array layout, first-label tie identity and Float64/tolerance behavior remain refinement questions
§24 schema	<code>SchInfluenceDiagram</code> (BayesianNetworks project), <code>lake.exe</code> <code>emit_schema</code>	<code>SchInfluenceDiagram</code> , compared with the emitted schema JSON

```
namespace InfluenceDiagramsProofs
```

```
-- Smoke lemma so that the axiom audit always has a first line. -/
theorem smoke : (1 : Nat) + 1 = 2 := rfl
```

```
end InfluenceDiagramsProofs
```

InfluenceDiagramsProofs.Finite.InfluenceDiagram

```
import BayesianNetworksProofs.Finite.BayesNet
import Mathlib.Algebra.Ring.Defs
import Mathlib.Algebra.BigOperators.Group.Finset.Piecewise
```

Concrete finite influence diagrams — the semantic model behind `InfluenceDiagrams.jl` (SPEC §23–§27, §36, §37).

A `FinInfluenceDiagram` extends the finite Bayesian-network shape `FinBayesNet` of the BayesianNetworks proofs project (chance variables V , chance mechanisms M , states, target, parents) with

- a finite type D of decisions, each controlling an action variable and reading an information set `info d` (the `Decision / InformationInput` rows of `SchInfluenceDiagram`);
- a finite type U of utility nodes, each reading a scope `uscope j` (the `Utility / UtilityInput` rows).

Kernels (chance), policies (decisions) and utilities live outside the structure, exactly as `kernel_ref / utility_ref` point outside the `ACSet` in Julia.

Validity (SPEC §37):

- `GeneratorsDisjoint` — no variable is both a mechanism target and an action (item 2);
- `GeneratorsCover` — every variable is a mechanism target or an action;
- `Closed` — `Sum.elim target action : M ⊗ D → V` is bijective, i.e. every variable has exactly one generator, mechanism or decision (items 1, 2 together with `closed_iff`);
- `IDOrder` — a topological order of the variables in which parents precede mechanism targets and information variables precede actions (items 4, 8, 11).

A `Policy id R d` is a normalised kernel `Assignment → states (action d) → R` that reads only `info d` (`LocalOn`) (SPEC §26). Over R , nonnegativity must be supplied separately for a stochastic interpretation (`Strategy.Nonneg` in `Optimization.lean`). A deterministic policy is `Policy.ofFun`, a constant one `Policy.const`. A `Strategy id R` is one policy per decision — complete by construction (SPEC §27).

```
set_option autoImplicit false
```

```

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

/-- A finite influence-diagram *shape*: a finite Bayesian-network shape plus decisions
(action variable, information set) and utility nodes (scope). -/
structure FinInfluenceDiagram extends FinBayesNet where
  /-- Decisions (one per 'Decision' row of the ACSet). -/
  D : Type
  [fintypeD : Fintype D]
  [decD : DecidableEq D]
  /-- The action variable controlled by each decision ('decision_variable'). -/
  action : D → V
  /-- The information set 'I_D' of each decision (the 'InformationInput' rows). -/
  info : D → Finset V
  /-- Utility nodes (one per 'Utility' row). -/
  U : Type
  [fintypeU : Fintype U]
  [decU : DecidableEq U]
  /-- The variables read by each utility node (the 'UtilityInput' rows). -/
  uscope : U → Finset V

attribute [instance] FinInfluenceDiagram.fintypeD FinInfluenceDiagram.decD
  FinInfluenceDiagram.fintypeU FinInfluenceDiagram.decU

namespace FinInfluenceDiagram

variable (id : FinInfluenceDiagram)

```

Generators and validity

```

/-- Every variable is generated by a mechanism or by a decision. -/
def generator : id.M ⊕ id.D → id.V := Sum.elim id.target id.action

@[simp] theorem generator_inl (m : id.M) : id.generator (.inl m) = id.target m := rfl

@[simp] theorem generator_inr (d : id.D) : id.generator (.inr d) = id.action d := rfl

/-- No variable is both a mechanism target and a decision action (SPEC §37 item 2: an
uninstantiated action has no chance mechanism). -/
def GeneratorsDisjoint : Prop := ∀ m d, id.target m ≠ id.action d

/-- Every variable is a mechanism target or a decision action. -/
def GeneratorsCover : Prop := ∀ v, (∃ m, id.target m = v) ∨ (∃ d, id.action d = v)

/-- Closed influence diagram: every variable has exactly one generator – a chance mechanism or a
decision ('validate' items 1 and 2 plus closedness of the chance part). -/
def Closed : Prop := Function.Bijective id.generator

/-- 'Closed' unpacked: both generator maps are injective, disjoint and jointly cover 'V'. -/
theorem closed_iff :
  id.Closed ↔ Function.Injective id.target ∧ Function.Injective id.action ∧
  id.GeneratorsDisjoint ∧ id.GeneratorsCover := by
  constructor
  · rintro ⟨hinj, hsurj⟩
    refine ⟨fun m m' h ⇒ ?_, fun d d' h ⇒ ?_, fun m d h ⇒ ?_, fun v ⇒ ?_⟩
    · exact Sum.inl_injective (hinj (a₁ := .inl m) (a₂ := .inl m') h)
    · exact Sum.inr_injective (hinj (a₁ := .inr d) (a₂ := .inr d') h)
    · exact Sum.inl_ne_inr (hinj (a₁ := .inl m) (a₂ := .inr d) h)
    · obtain ⟨g, hg⟩ := hsurj v
      cases g with
      | inl m ⇒ exact Or.inl ⟨m, hg⟩
      | inr d ⇒ exact Or.inr ⟨d, hg⟩
  · rintro ⟨hT, hA, hdisj, hcover⟩

```

```

refine ⟨fun g g' h ⇒ ?_, fun v ⇒ ?_⟩
· cases g with
| inl m ⇒
  cases g' with
  | inl m' ⇒ exact congrArg Sum.inl (hT h)
  | inr d' ⇒ exact absurd h (hdisj m d')
| inr d ⇒
  cases g' with
  | inl m' ⇒ exact absurd h.symm (hdisj m' d)
  | inr d' ⇒ exact congrArg Sum.inr (hA h)
· rcases hcover v with ⟨m, hm⟩ | ⟨d, hd⟩
· exact ⟨.inl m, hm⟩
· exact ⟨.inr d, hd⟩

/-- Build 'Closed' from its four components. -/
theorem Closed.of (hT : Function.Injective id.target) (hA : Function.Injective id.action)
  (hdisj : id.GeneratorsDisjoint) (hcover : id.GeneratorsCover) : id.Closed :=
  id.closed_iff.2 ⟨hT, hA, hdisj, hcover⟩

/-- A topological order of the influence diagram: a duplicate-free complete list of the variables
in which every parent of a mechanism precedes its target and every information variable of a
decision precedes its action; no generator reads the variable it generates (SPEC §37 items 4,
8, 11; the decision precedence is the induced order of the actions). -/
structure IDOrder where
  order : List id.V
  nodup : order.Nodup
  complete : ∀ v, v ∈ order
  /-- For 'a' before 'b', 'b' is not a parent of the mechanism generating 'a'. -/
  parents_before : order.Pairwise (fun a b ⇒ ∀ m, id.target m = a → b ∉ id.parents m)
  /-- For 'a' before 'b', 'b' is not an information variable of the decision controlling 'a'
  (information is available before the decision is taken). -/
  info_before_action : order.Pairwise (fun a b ⇒ ∀ d, id.action d = a → b ∉ id.info d)
  no_self : ∀ m, id.target m ∉ id.parents m
  no_self_info : ∀ d, id.action d ∉ id.info d

```

Policies and strategies

```

variable {id} {R : Type} [CommSemiring R]

/-- A policy for decision 'd': a normalised kernel from assignments to actions that reads only
the information set 'info d' (SPEC §26, §37 item 9). Real-valued policies may be signed unless
nonnegativity is required separately. -/
structure Policy (id : FinInfluenceDiagram) (R : Type) [CommSemiring R] (d : id.D) where
  /-- 'kernel x a' is the weight of action 'a' given assignment 'x'. -/
  kernel : id.Assignment → id.states (id.action d) → R
  /-- The policy reads only its information set. -/
  localOn : LocalOn (id.info d) kernel
  /-- Action weights sum to one; nonnegativity is a separate condition over 'R'. -/
  normalised : ∀ x, ∑ a, kernel x a = 1

/-- A complete strategy: one policy per decision (SPEC §27, §37 item 10). -/
abbrev Strategy (id : FinInfluenceDiagram) (R : Type) [CommSemiring R] :=
  (d : id.D) → Policy id R d

namespace Policy

variable {d : id.D}

/-- A deterministic policy 'δ : I_D → A_D', given as a function of the whole assignment that
depends only on the information set. -/
def ofFun (f : id.Assignment → id.states (id.action d))
  (hf : ∀ x x' : id.Assignment, (∀ p ∈ id.info d, x p = x' p) → f x = f x') : Policy id R d where
  kernel x a := if a = f x then 1 else 0
  localOn x x' h := by

```

```

    funext a
    dsimp only
    rw [hf x x' h]
    normalised x := by simp

@[simp] theorem ofFun_kernel (f : id.Assignment → id.states (id.action d)) (hf)
  (x : id.Assignment) (a : id.states (id.action d)) :
  (ofFun (R := R) f hf).kernel x a = if a = f x then 1 else 0 := rfl

/-- The constant policy always choosing the action 'a' ('fix_decision', SPEC §41). -/
def const (d : id.D) (a : id.states (id.action d)) : Policy id R d where
  kernel _ b := if b = a then 1 else 0
  localOn _ _ := rfl
  normalised _ := by simp

@[simp] theorem const_kernel (d : id.D) (a : id.states (id.action d)) (x : id.Assignment)
  (b : id.states (id.action d)) : (const (R := R) d a).kernel x b = if b = a then 1 else 0 := rfl

/-- The constant policy reads no variable at all: it is local for every information set. -/
theorem const_localOn (d : id.D) (a : id.states (id.action d)) (P : Finset id.V) :
  LocalOn P (const (R := R) d a).kernel := fun _ _ => rfl

/-- In particular it is local for the empty information set. -/
theorem const_localOn_empty (d : id.D) (a : id.states (id.action d)) :
  LocalOn ∅ (const (R := R) d a).kernel :=
  const_localOn d a ∅

end Policy

namespace Strategy

/-- 'fix_decision': replace the policy of 'd' by the constant policy at 'a' (SPEC §41). -/
def fix (σ : Strategy id R) (d : id.D) (a : id.states (id.action d)) : Strategy id R :=
  Function.update σ d (Policy.const d a)

@[simp] theorem fix_self (σ : Strategy id R) (d : id.D) (a : id.states (id.action d)) :
  σ.fix d a d = Policy.const d a := by
  simp [fix]

theorem fix_of_ne (σ : Strategy id R) {d d' : id.D} (h : d' ≠ d) (a : id.states (id.action d)) :
  σ.fix d a d' = σ d' := by
  simp [fix, Function.update_of_ne h]

end Strategy

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.Instantiate

```

import InfluenceDiagramsProofs.Finite.InfluenceDiagram
import BayesianNetworksProofs.Finite.Evaluation
import Mathlib.Data.Fintype.Sum
import Mathlib.Data.Fintype.BigOperators

```

Proposition 5 (policy instantiation) for the concrete finite model (SPEC §28, §61).

`instantiate id` is the Bayesian-network shape obtained by turning every decision into a mechanism: mechanisms $M \oplus D$, targets `Sum.elim target action`, parents `Sum.elim parents info` (the `PolicyRef` mechanisms that Julia's `instantiate(id, σ)` adds before Δ -migrating into a fresh `BayesNet`). The strategy σ supplies the kernels of the new mechanisms: `strategyKernel κ σ = Sum.elim κ (fun d => (σ d).kernel)`.

- `closed_instantiate` — the instantiated network is closed iff the influence diagram is `(Sum.elim target`

- action bijective);
- local_instantiate, normalised_instantiate — the instantiated family is local and normalised when the chance kernels are, because policies are local on their information set and normalised by definition;
- IDOrder.toTopoOrder — an order of the influence diagram is a topological order of the instantiated network;
- hence sum_joint_instantiate_eq_one (Proposition 1 of the BN project applied to the instantiated network): $\sum x, \text{joint}(\text{strategyKernel } \kappa \ \sigma) \ x = 1$, the law P_σ of SPEC §31;
- joint_instantiate — $P_\sigma(x) = (\prod m, \kappa_m) * (\prod d, \delta_d)$, SPEC §31's factorisation.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

namespace FinInfluenceDiagram

/-- The Bayesian-network shape with one mechanism per chance variable *and* per decision.
Reducible so that '(id.instantiate).Assignment' is seen to be 'id.Assignment'. -/
@[reducible] def instantiate (id : FinInfluenceDiagram) : FinBayesNet where
  V := id.V
  M := id.M ⊕ id.D
  states := id.states
  target := Sum.elim id.target id.action
  parents := Sum.elim id.parents id.info

variable {id : FinInfluenceDiagram} {R : Type}

@[simp] theorem instantiate_target_inl (m : id.M) : id.instantiate.target (.inl m) = id.target m := rfl

@[simp] theorem instantiate_target_inr (d : id.D) : id.instantiate.target (.inr d) = id.action d := rfl

@[simp] theorem instantiate_parents_inl (m : id.M) :
  id.instantiate.parents (.inl m) = id.parents m := rfl

@[simp] theorem instantiate_parents_inr (d : id.D) :
  id.instantiate.parents (.inr d) = id.info d := rfl

/-- **Proposition 5 (shape).** The instantiated network is closed exactly when every variable of
the influence diagram has one generator. -/
theorem closed_instantiate_iff : id.instantiate.Closed ↔ id.Closed := Iff.rfl

theorem closed_instantiate (h : id.Closed) : id.instantiate.Closed := h

/-- **Proposition 5 (order).** An order of the influence diagram is a topological order of the
instantiated network: parents precede mechanism targets and information precedes actions. -/
def IDOrder.toTopoOrder (ord : id.IDOrder) : id.instantiate.TopoOrder where
  order := ord.order
  nodup := ord.nodup
  complete := ord.complete
  parents_before := (ord.parents_before.and ord.info_before_action).imp fun {a b} h => by
    rintro (m | d) hm
    · exact h.1 m hm
    · exact h.2 d hm
  no_self := fun
    | .inl m => ord.no_self m
    | .inr d => ord.no_self_info d

variable [CommSemiring R]

/-- The kernels of the instantiated network: chance kernels for the chance mechanisms, policy
kernels for the decisions. -/
def strategyKernel (κ : id.Kernel R) (σ : Strategy id R) : id.instantiate.Kernel R := fun
  | .inl m => κ m
  | .inr d => (σ d).kernel

```

```

@[simp] theorem strategyKernel_inl (κ : id.Kernel R) (σ : Strategy id R) (m : id.M) :
  strategyKernel κ σ (.inl m) = κ m := rfl

@[simp] theorem strategyKernel_inr (κ : id.Kernel R) (σ : Strategy id R) (d : id.D) :
  strategyKernel κ σ (.inr d) = (σ d).kernel := rfl

/-- Proposition 5 (locality). Local chance kernels and policies (local on their information
sets by definition) give a local family for the instantiated network. -/
theorem local_instantiate (κ : id.Kernel R) (σ : Strategy id R) (hloc : ∀ m, Local κ m) :
  ∀ m, Local (strategyKernel κ σ) m
| .inl m ⇒ hloc m
| .inr d ⇒ (σ d).localOn

/-- Proposition 5 (normalisation). Normalised chance kernels and policies give a normalised
family for the instantiated network. -/
theorem normalised_instantiate (κ : id.Kernel R) (σ : Strategy id R)
  (hnorm : ∀ m, Normalised κ m) : ∀ m, Normalised (strategyKernel κ σ) m
| .inl m ⇒ hnorm m
| .inr d ⇒ (σ d).normalised

/-- Proposition 5. For a closed, acyclic influence diagram with local normalised chance
kernels, every complete strategy yields a Bayesian network whose joint is a probability
distribution – the law 'P_σ' of SPEC §31. -/
theorem sum_joint_instantiate_eq_one (κ : id.Kernel R) (σ : Strategy id R) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m) :
  ∑ x, joint (strategyKernel κ σ) x = 1 :=
  sum_joint_eq_one (strategyKernel κ σ) (closed_instantiate hclosed) ord.toTopoOrder
  (local_instantiate κ σ hloc) (normalised_instantiate κ σ hnorm)

/-- The joint of the instantiated network factorises as chance kernels times policy kernels
(SPEC §31, 'P_σ(x, a) = ∏ κ_X(x_X | pa_X) ∏ δ_D(a_D | i_D)'). -/
theorem joint_instantiate (κ : id.Kernel R) (σ : Strategy id R) (x : id.Assignment) :
  joint (strategyKernel κ σ) x =
  (∏ m, κ m x (x (id.target m))) * ∏ d, (σ d).kernel x (x (id.action d)) := by
  unfold joint
  exact Fintype.prod_sum_type _

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.ExpectedUtility

```

import InfluenceDiagramsProofs.Finite.Instantiate
import BayesianNetworksProofs.Finite.Intervention

```

Proposition 6 (expected utility) for the concrete finite model (SPEC §29–§31, §41, §61).

Utilities are deterministic functionals $u_j : \text{Assignment} \rightarrow \mathbb{R}$, one per utility node, each reading only its scope (Utility.Local); the first implementation aggregates additively, $\text{totalUtility } u \ x = \sum j, u_j \ x$ (SPEC §30). The expected utility of a strategy is the expectation of the total utility under the law P_σ of the instantiated network (SPEC §31):

$\text{expectedUtility } \kappa \ \sigma \ u = \sum x, \text{joint } (\text{strategyKernel } \kappa \ \sigma) \ x * \text{totalUtility } u \ x.$

- `expectedUtility_add`, `expectedUtility_smul` — EU is linear in the utility;
- `expectedUtility_eq_sum` — additive decomposition: $\text{EU} = \sum j, \sum x, P_\sigma(x) * u_j \ x$;
- `expectedUtility_const` — a constant total utility c has expected utility c (Proposition 5's normalisation);
- `fix_decision`: `strategyKernel_fix_eq_intervene` — fixing decision d to a in the strategy is the hard intervention `do(action d = a)` on the instantiated network (Proposition 4 of the BN project), so `joint_fix` gives the truncated factorisation and `expectedUtility_fix` the expected utility of a fixed decision.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

namespace FinInfluenceDiagram

variable {id : FinInfluenceDiagram} {R : Type}

```

Utilities

```

/-- A family of utility functionals, one per utility node ('utility_ref' in the ACSet). -/
abbrev Utility (id : FinInfluenceDiagram) (R : Type) := id.U → id.Assignment → R

/-- 'u j' depends on the assignment only through the scope of 'j' (its 'UtilityInput' rows). -/
def Utility.Local (u : Utility id R) (j : id.U) : Prop :=
  ∀ x x' : id.Assignment, (∀ p ∈ id.uscope j, x p = x' p) → u j x = u j x'

variable [CommSemiring R]

/-- Additive aggregation 'U(x) = ∑ j, u j (x_{scope j})' (SPEC §30). -/
def totalUtility (u : Utility id R) (x : id.Assignment) : R := ∑ j, u j x

/-- Expected utility of a strategy: 'EU(σ) = ∑ x, P_σ(x) U(x)' (SPEC §31; the Julia
'expected_utility(id, strategy)' reference algorithm). -/
def expectedUtility (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) : R :=
  ∑ x, joint (strategyKernel κ σ) x * totalUtility u x

/-- **Proposition 6.** 'EU(σ) = E_{P_σ}[U]': the expectation of the total utility under the
instantiated joint, written out via 'joint_instantiate'. -/
theorem expectedUtility_eq (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  expectedUtility κ σ u =
    ∑ x, ((∏ m, κ m x (x (id.target m))) * ∏ d, (σ d).kernel x (x (id.action d))) *
      ∑ j, u j x := by
  unfold expectedUtility totalUtility
  exact Finset.sum_congr rfl fun x _ => by rw [joint_instantiate]

```

Linearity

```

theorem totalUtility_add (u₁ u₂ : Utility id R) (x : id.Assignment) :
  totalUtility (u₁ + u₂) x = totalUtility u₁ x + totalUtility u₂ x := by
  simp [totalUtility, Finset.sum_add_distrib]

theorem totalUtility_smul (c : R) (u : Utility id R) (x : id.Assignment) :
  totalUtility (fun j y => c * u j y) x = c * totalUtility u x := by
  simp [totalUtility, Finset.mul_sum]

/-- Expected utility is additive in the utility. -/
theorem expectedUtility_add (κ : id.Kernel R) (σ : Strategy id R) (u₁ u₂ : Utility id R) :
  expectedUtility κ σ (u₁ + u₂) = expectedUtility κ σ u₁ + expectedUtility κ σ u₂ := by
  unfold expectedUtility
  rw [← Finset.sum_add_distrib]
  exact Finset.sum_congr rfl fun x _ => by rw [totalUtility_add, mul_add]

/-- Expected utility is homogeneous in the utility. -/
theorem expectedUtility_smul (κ : id.Kernel R) (σ : Strategy id R) (c : R) (u : Utility id R) :
  expectedUtility κ σ (fun j y => c * u j y) = c * expectedUtility κ σ u := by
  unfold expectedUtility
  rw [Finset.mul_sum]
  exact Finset.sum_congr rfl fun x _ => by rw [totalUtility_smul, mul_left_comm]

/-- Additive decomposition: the expected utility is the sum of the expected utilities of the

```

```

individual utility nodes (SPEC §30-§31). -/
theorem expectedUtility_eq_sum (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  expectedUtility κ σ u =  $\sum j, \sum x, \text{joint } (\text{strategyKernel } \kappa \sigma) x * u j x$  := by
  unfold expectedUtility totalUtility
  rw [Finset.sum_comm]
  exact Finset.sum_congr rfl fun x _ => Finset.mul_sum _ _

/-- A utility whose total is the constant 'c' has expected utility 'c' (uses Proposition 5:
the instantiated joint sums to one). -/
theorem expectedUtility_const (κ : id.Kernel R) (σ : Strategy id R) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc :  $\forall m, \text{Local } \kappa m$ ) (hnorm :  $\forall m, \text{Normalised } \kappa m$ )
  (u : Utility id R) (c : R) (hu :  $\forall x, \text{totalUtility } u x = c$ ) :
  expectedUtility κ σ u = c := by
  unfold expectedUtility
  simp_rw [hu]
  rw [← Finset.sum_mul, sum_joint_instantiate_eq_one κ σ hclosed ord hloc hnorm, one_mul]

```

Fixing a decision

```

/-- 'fix_decision' is a hard intervention.** Replacing the policy of 'd' by the constant policy
at 'a' gives exactly the kernel family 'do(action d = a)' of the instantiated network
('intervene' of the BN project on the mechanism 'inr d'). -/
theorem strategyKernel_fix_eq_intervene (κ : id.Kernel R) (σ : Strategy id R) (d : id.D)
  (a : id.states (id.action d)) :
  strategyKernel κ (σ.fix d a) = intervene (strategyKernel κ σ) (.inr d) a := by
  funext m
  cases m with
  | inl m =>
    rw [intervene_of_ne _ Sum.inl_ne_inr]
    rfl
  | inr d' =>
    by_cases h : d' = d
    · subst h
      funext x y
      show (σ.fix d' a d').kernel x y = _
      rw [Strategy.fix_self, Policy.const_kernel, intervene_self]
      rfl
    · rw [intervene_of_ne _ (fun h' => h (Sum.inr_injective h'))]
      show (σ.fix d a d').kernel = (σ d').kernel
      rw [Strategy.fix_of_ne σ h]

/-- The joint under a fixed decision: the truncated factorisation of Proposition 4. -/
theorem joint_fix (κ : id.Kernel R) (σ : Strategy id R) (d : id.D) (a : id.states (id.action d))
  (x : id.Assignment) :
  joint (strategyKernel κ (σ.fix d a)) x =
    (if x (id.action d) = a then 1 else 0) *
     $\prod m \in \text{Finset.univ.erase } (\text{Sum.inr } d), \text{strategyKernel } \kappa \sigma m x (x (\text{id.instantiate.target } m))$  :=
  by
  rw [strategyKernel_fix_eq_intervene]
  exact joint_intervene (strategyKernel κ σ) (.inr d) a x

/-- Expected utility of a fixed decision, via the truncated factorisation. -/
theorem expectedUtility_fix (κ : id.Kernel R) (σ : Strategy id R) (d : id.D)
  (a : id.states (id.action d)) (u : Utility id R) :
  expectedUtility κ (σ.fix d a) u =
     $\sum x, (\text{if } x (\text{id.action } d) = a \text{ then } 1 \text{ else } 0) * (\prod m \in \text{Finset.univ.erase } (\text{Sum.inr } d), \text{strategyKernel } \kappa \sigma m x (x (\text{id.instantiate.target } m))) * \text{totalUtility } u x$  := by
  unfold expectedUtility
  exact Finset.sum_congr rfl fun x _ => by rw [joint_fix]

end FinInfluenceDiagram
end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.Information

```
import InfluenceDiagramsProofs.Finite.ExpectedUtility
```

Information monotonicity (SPEC §34, §35, §55.7).

An information arc $X \rightarrow D$ enlarges the admissible policy class and nothing else: a policy that reads only $\text{info } d$ also reads only a superset of it (LocalOn is monotone), so every strategy of an influence diagram is a strategy of the diagram with enlarged information sets, with the *same* kernels and hence the same expected utility. Consequently the optimal expected utility (supremum over strategies) cannot decrease when information is added — the requirement of SPEC §55.7 and the sign of the expected value of information (SPEC §35).

```
set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

/-- Locality is monotone in the set of variables read. -/
theorem localOn_mono {bn : FinBayesNet} {R : Type} {P Q : Finset bn.V} (hPQ : P ⊆ Q) {v : bn.V}
  {k : bn.Assignment → bn.states v → R} (hk : LocalOn P k) : LocalOn Q k :=
  fun x x' h => hk x x' fun p hp => h p (hpQ hp)

namespace FinInfluenceDiagram

variable {id : FinInfluenceDiagram} {R : Type} [CommSemiring R]

/-- The same influence diagram with the information sets replaced by 'info'. Reducible so that
its decisions, variables and assignments are seen to be those of 'id'. -/
@[reducible] def withInfo (id : FinInfluenceDiagram) (info' : id.D → Finset id.V) :
  FinInfluenceDiagram :=
  { id with info := info' }

@[simp] theorem withInfo_info (info' : id.D → Finset id.V) (d : id.D) :
  (id.withInfo info').info d = info' d := rfl

/-- A policy for an information set is a policy for any larger one. -/
def Policy.enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d) {d : id.D}
  (π : Policy id R d) : Policy (id.withInfo info') R d where
  kernel := π.kernel
  localOn := localOn_mono (h d) π.localOn
  normalised := π.normalised

/-- A strategy of 'id' is a strategy of 'id' with enlarged information sets. -/
def Strategy.enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (σ : Strategy id R) : Strategy (id.withInfo info') R :=
  fun d => (σ d).enlarge h

/-- Enlarging the information sets does not change the kernels, hence not the joint. -/
theorem strategyKernel_enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (κ : id.Kernel R) (σ : Strategy id R) :
  strategyKernel (id := id.withInfo info') κ (σ.enlarge h) = strategyKernel κ σ := by
  funext m
  cases m <|> rfl

/-- Enlarging the information sets does not change the expected utility of a strategy. -/
theorem expectedUtility_enlarge {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  expectedUtility (id := id.withInfo info') κ (σ.enlarge h) u = expectedUtility κ σ u := by
  unfold expectedUtility
  rw [strategyKernel_enlarge]
  rfl

/-- **Information monotonicity** (SPEC §55.7). Every strategy of an influence diagram is matched,
with equal expected utility, by a strategy of the diagram with enlarged information sets; so the
optimal expected utility cannot decrease when information is added. -/
```

```

theorem exists_strategy_enlarged_eq {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R) :
  ∃ σ' : Strategy (id.withInfo info') R,
    expectedUtility (id := id.withInfo info') κ σ' u = expectedUtility κ σ u :=
  ⟨σ.enlarge h, expectedUtility_enlarge h κ σ u⟩

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.Optimization

```

import InfluenceDiagramsProofs.Finite.ExpectedUtility
import Mathlib.Data.Real.Basic
import Mathlib.Data.Fintype.BigOperators
import Mathlib.Data.Finset.Max
import Mathlib.Algebra.Order.BigOperators.Group.Finset
import Mathlib.Algebra.Order.BigOperators.GroupWithZero.Finset
import Mathlib.Tactic.NormNum

```

Deterministic global optimality for any finite number of decisions (SPEC §26–§27, §32, and the deterministic-search prerequisite of §61 Proposition 7).

A nonnegative strategy is a convex mixture of deterministic policy tables: independently draw an action for every information configuration of every decision. Finite distributivity shows that mixing the induced joints recovers exactly the original joint. Expected utility is therefore a convex combination of the deterministic expected utilities and cannot exceed their finite maximum.

Perfect recall is not needed for this global existence theorem. The former Roadmap prose conflated deterministic sufficiency with the correctness of greedy/strong-order decision elimination. The latter is **not** proved here. This result gives no efficient search algorithm: the table space can be exponentially large.

All action spaces are nonempty because `FinBayesNet.nonemptyS` is part of the inherited model. Nonnegativity of the *strategy* is essential for the convex bound; normalisation is a `Policy` field. Chance kernels need no positivity, locality or normalisation for the algebraic identity and the optimisation bound. To interpret the joint as a probability law, the validity, locality, normalisation and positivity hypotheses must be supplied separately.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

namespace FinInfluenceDiagram

variable {id : FinInfluenceDiagram}

/-- Every policy in the strategy is a deterministic local function. -/
def Strategy.Deterministic (σ : Strategy id R) : Prop :=
  ∀ d, ∃ f hf, σ d = Policy.ofFun f hf

/-- Policy normalisation alone does not prohibit signed entries; this condition does. -/
def Strategy.Nonneg (σ : Strategy id R) : Prop :=
  ∀ d x a, 0 ≤ (σ d).kernel x a

/-- Values of precisely the variables visible to a decision. -/
abbrev InfoAssignment (id : FinInfluenceDiagram) (d : id.D) :=
  (v : {v : id.V // v ∈ id.info d}) → id.states v.val

/-- Restrict a complete assignment to the decision's information variables. -/
def infoAssignment (d : id.D) (x : id.Assignment) : InfoAssignment id d :=
  fun v => x v.val

/-- Complete an information assignment with arbitrary states off the information set.

```

```

These states exist by the explicit 'nonemptyS' fields of the finite model. -/
noncomputable def extendInfo (d : id.D) (t : InfoAssignment id d) : id.Assignment :=
  fun v => if hv : v ∈ id.info d then t ⟨v, hv⟩ else Classical.choice (id.nonemptyS v)

theorem policy_extendInfo (d : id.D) (p : Policy id R d) (x : id.Assignment) :
  p.kernel (extendInfo d (infoAssignment d x)) = p.kernel x := by
  apply p.localOn
  intro v hv
  simp [extendInfo, infoAssignment, hv]

/-- One action per information row, for every decision. This is a finite, nonempty type. -/
abbrev PolicyTable (id : FinInfluenceDiagram) :=
  (d : id.D) → InfoAssignment id d → id.states (id.action d)

/-- Interpret a complete policy table as a local deterministic strategy. -/
def tableStrategy (t : PolicyTable id) : Strategy id R :=
  fun d => Policy.ofFun (fun x => t d (infoAssignment d x)) (by
    intro x y h
    apply congrArg (t d)
    funext v
    exact h v.val v.property)

theorem tableStrategy_deterministic (t : PolicyTable id) :
  (tableStrategy t).Deterministic := by
  intro d
  exact ⟨_, _, rfl⟩

theorem tableStrategy_nonneg (t : PolicyTable id) : (tableStrategy t).Nonneg := by
  intro d x a
  change 0 ≤ if a = t d (infoAssignment d x) then (1 : R) else 0
  split_ifs
  · exact zero_le_one
  · exact le_refl 0

section FiniteMixture

variable {I A : Type} [Fintype I] [DecidableEq I] [Fintype A] [DecidableEq A]

/-- A product distribution over tables recovers any one of its rows. -/
theorem sum_table_indicator (q : I → A → R) (hq : ∀ i, ∑ a, q i a = 1)
  (i : I) (a : A) :
  (∑ f : I → A, (∏ j, q j (f j)) * (if a = f i then 1 else 0)) = q i a := by
  calc
  - = ∑ f : I → A, ∏ j, q j (f j) *
    (if j = i then (if a = f j then 1 else 0) else 1) := by
    refine Finset.sum_congr rfl fun f _ => ?_
    rw [Finset.prod_mul_distrib]
    simp
  - = ∏ j, ∑ b, q j b * (if j = i then (if a = b then 1 else 0) else 1) :=
    (Fintype.prod_sum (fun (j : I) (b : A) =>
      q j b * (if j = i then (if a = b then 1 else 0) else 1))).symm
  - = q i a := by
    have hs : ∀ j, (∑ b, q j b * (if j = i then (if a = b then 1 else 0) else 1)) =
      if j = i then q j a else 1 := by
      intro j
      by_cases h : j = i
      · simp [h, mul_ite]
      · simp [h, hq]
    simp_rw [hs]
    simp

end FiniteMixture

/-- Probability of drawing one local deterministic policy table from a policy. -/
noncomputable def policyWeight (d : id.D) (p : Policy id R d)
  (t : InfoAssignment id d → id.states (id.action d)) : R :=

```

```

    Π i, p.kernel (extendInfo d i) (t i)

theorem sum_policyWeight (d : id.D) (p : Policy id ℝ d) :
  ∑ t, policyWeight d p t = 1 := by
  classical
  unfold policyWeight
  rw [← Fintype.prod_sum]
  simp [p.normalised]

theorem policyWeight_indicator (d : id.D) (p : Policy id ℝ d)
  (x : id.Assignment) (a : id.states (id.action d)) :
  (∑ t, policyWeight d p t * (if a = t (infoAssignment d x) then 1 else 0)) =
    p.kernel x a := by
  classical
  unfold policyWeight
  rw [sum_table_indicator _ (fun i ⇒ p.normalised (extendInfo d i)),
    policy_extendInfo]

/-- Independent table draws for all decisions. -/
noncomputable def tableWeight (σ : Strategy id ℝ) (t : PolicyTable id) : ℝ :=
  Π d, policyWeight d (σ d) (t d)

theorem sum_tableWeight (σ : Strategy id ℝ) : ∑ t, tableWeight σ t = 1 := by
  classical
  unfold tableWeight
  rw [← Fintype.prod_sum]
  simp [sum_policyWeight]

theorem tableWeight_nonneg (σ : Strategy id ℝ) (hσ : σ.Nonneg) (t : PolicyTable id) :
  0 ≤ tableWeight σ t := by
  unfold tableWeight policyWeight
  exact Finset.prod_nonneg fun d _ ⇒ Finset.prod_nonneg fun i _ ⇒ hσ d _ _

/-- Mixing deterministic-table joints recovers the original joint exactly. -/
theorem joint_table_mixture (κ : id.Kernel ℝ) (σ : Strategy id ℝ) (x : id.Assignment) :
  joint (strategyKernel κ σ) x =
    ∑ t : PolicyTable id, tableWeight σ t * joint (strategyKernel κ (tableStrategy t)) x := by
  classical
  simp_rw [joint_instantiate]
  simp only [tableWeight, tableStrategy, Policy.ofFun_kernel]
  simp_rw [mul_left_comm (Π d, policyWeight d (σ d) _) (Π m, κ m x (x (id.target m)))]
  rw [← Finset.mul_sum]
  congr 1
  simp only [← Finset.prod_mul_distrib]
  calc
    _ = Π d, ∑ t : InfoAssignment id d → id.states (id.action d),
      policyWeight d (σ d) t *
        (if x (id.action d) = t (infoAssignment d x) then 1 else 0) :=
      Finset.prod_congr rfl fun d _ ⇒ (policyWeight_indicator d (σ d) x _).symm
    _ = _ := Fintype.prod_sum (fun d (t : InfoAssignment id d → id.states (id.action d)) ⇒
      policyWeight d (σ d) t * (if x (id.action d) = t (infoAssignment d x) then 1 else 0))

/-- Expected utility is a convex combination of deterministic-table expected utilities when
the strategy is nonnegative. The identity itself does not need nonnegativity. -/
theorem expectedUtility_table_mixture (κ : id.Kernel ℝ) (σ : Strategy id ℝ)
  (u : Utility id ℝ) :
  expectedUtility κ σ u =
    ∑ t : PolicyTable id, tableWeight σ t * expectedUtility κ (tableStrategy t) u := by
  classical
  unfold expectedUtility
  simp_rw [joint_table_mixture κ σ, Finset.sum_mul]
  rw [Finset.sum_comm]
  refine Finset.sum_congr rfl fun t _ ⇒ ?_
  rw [Finset.mul_sum]
  exact Finset.sum_congr rfl fun x _ ⇒ mul_assoc _ _ _

```

```

/-- **Global deterministic sufficiency for all finite diagrams, including limited memory.**
This is exhaustive-table optimality, not correctness of decision variable elimination. -/
theorem exists_deterministic_optimal_all (κ : id.Kernel ℝ) (u : Utility id ℝ) :
  ∃ σ' : Strategy id ℝ, σ'.Deterministic ∧ σ'.Nonneg ∧
    ∀ σ : Strategy id ℝ, σ.Nonneg → expectedUtility κ σ u ≤ expectedUtility κ σ' u := by
  classical
  obtain ⟨t, _, ht⟩ := Finset.exists_max_image (Finset.univ : Finset (PolicyTable id))
    (fun t ⇒ expectedUtility κ (tableStrategy t) u) Finset.univ_nonempty
  refine ⟨tableStrategy t, tableStrategy_deterministic t, tableStrategy_nonneg t, ?_⟩
  intro σ hσ
  rw [expectedUtility_table_mixture]
  calc
    _ ≤ ∑ s : PolicyTable id, tableWeight σ s * expectedUtility κ (tableStrategy t) u :=
      Finset.sum_le_sum fun s hs ⇒
        mul_le_mul_of_nonneg_left (ht s hs) (tableWeight_nonneg σ hσ s)
    _ = _ := by rw [← Finset.sum_mul, sum_tableWeight, one_mul]

/-- The original single-decision Roadmap claim, unchanged in semantic strength. Its 'Unique'
and nonnegative-chance-kernel premises are sufficient but unnecessary. -/
theorem exists_deterministic_optimal [Unique id.D] (κ : id.Kernel ℝ)
  (_hκ : ∀ m x y, 0 ≤ κ m x y) (u : Utility id ℝ) :
  ∃ σ' : Strategy id ℝ, σ'.Deterministic ∧
    ∀ σ : Strategy id ℝ, σ.Nonneg → expectedUtility κ σ u ≤ expectedUtility κ σ' u := by
  obtain ⟨σ', hd, _, hopt⟩ := exists_deterministic_optimal_all κ u
  exact ⟨σ', hd, hopt⟩

/-- Normalisation without nonnegativity does not give a convex bound: signed weights
'(-1, 2)' sum to one but turn utilities '(0, 1)' into value two. -/
theorem signed_weights_exceed_every_action :
  ∃ (p u : Bool → ℝ), (∑ a, p a = 1) ∧ (∀ a, u a ≤ 1) ∧
    1 < ∑ a, p a * u a := by
  refine ⟨(fun a ⇒ if a then 2 else -1), (fun a ⇒ if a then 1 else 0), ?_, ?_, ?_⟩
  · norm_num [Fintype.sum_bool]
  · intro a
    cases a <|> norm_num
  · norm_num [Fintype.sum_bool]

end FinInfluenceDiagram

end InfluenceDiagramsProofs

```

InfluenceDiagramsProofs.Finite.OptimalInformation

```

import InfluenceDiagramsProofs.Finite.Information
import InfluenceDiagramsProofs.Finite.Optimization

```

Monotonicity of the attained optimal value, not only existence of an equal-value enlarged strategy (SPEC §34–§35, §55.7).

The finite optimum from Optimization.lean makes the optimal value a real number attained by a nonnegative deterministic strategy. Enlarging information preserves all old strategies, their nonnegativity and expected utilities. Hence the new optimal value is at least the old one, and their difference is nonnegative.

Chance kernels and utilities are held fixed. No acquisition cost is introduced. This is a statement about the admissible strategy classes, not Julia’s solver, automatic propagation of no-forgetting arcs, or the validity of an information enlargement as a temporal diagram.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.FinInfluenceDiagram

variable {id : FinInfluenceDiagram}

/-- The attained finite optimal expected utility (defined by choice of a proved maximizer). -/

```

```

noncomputable def optimalValue (κ : id.Kernel ℝ) (u : Utility id ℝ) : ℝ :=
  expectedUtility κ (Classical.choose (exists_deterministic_optimal_all κ u)) u

theorem optimalValue_attained (κ : id.Kernel ℝ) (u : Utility id ℝ) :
  ∃ σ : Strategy id ℝ, σ.Deterministic ∧ σ.Nonneg ∧
    expectedUtility κ σ u = optimalValue κ u := by
  refine ⟨Classical.choose (exists_deterministic_optimal_all κ u), ?_, ?_, rfl⟩
  · exact (Classical.choose_spec (exists_deterministic_optimal_all κ u)).1
  · exact (Classical.choose_spec (exists_deterministic_optimal_all κ u)).2.1

theorem expectedUtility_le_optimalValue (κ : id.Kernel ℝ) (u : Utility id ℝ)
  (σ : Strategy id ℝ) (hσ : σ.Nonneg) : expectedUtility κ σ u ≤ optimalValue κ u :=
  (Classical.choose_spec (exists_deterministic_optimal_all κ u)).2.2 σ hσ

/-- Optimal information monotonicity. Both sides are attained maxima, not unproved suprema. -/
theorem optimalValue_info_mono {info' : id.D → Finset id.V} (h : ∀ d, id.info d ⊆ info' d)
  (κ : id.Kernel ℝ) (u : Utility id ℝ) :
  optimalValue κ u ≤ optimalValue (id := id.withInfo info') κ u := by
  obtain ⟨σ, _, hσ, heq⟩ := optimalValue_attained κ u
  have hen : (σ.enlarge h).Nonneg := hσ
  rw [← heq, ← expectedUtility_enlarge h κ σ u]
  exact expectedUtility_le_optimalValue (id := id.withInfo info') κ u (σ.enlarge h) hen

/-- Cost-free expected value of information is nonnegative for the exact finite optimum. -/
theorem optimal_information_value_nonneg {info' : id.D → Finset id.V}
  (h : ∀ d, id.info d ⊆ info' d) (κ : id.Kernel ℝ) (u : Utility id ℝ) :
  0 ≤ optimalValue (id := id.withInfo info') κ u - optimalValue κ u :=
  sub_nonneg.mpr (optimalValue_info_mono h κ u)

end InfluenceDiagramsProofs.FinInfluenceDiagram

```

Exact probability/utility valuations

```

import InfluenceDiagramsProofs.Finite.OptimalInformation
import BayesianNetworksProofs.Finite.VariableElimination
import Mathlib.Tactic.Ring
import Mathlib.Algebra.Order.BigOperators.Ring.Finset

```

Source: InfluenceDiagrams.jl/src/valuation.jl, frozen in the second review packet. Combination multiplies probability potentials and adds divided utilities. Chance elimination uses a weighted sum divided by its mass, with zero denominator giving zero. Nonnegative probabilities prove that a zero denominator also has zero weighted numerator.

Scopes are sufficient dependency sets, not array layouts. The decision operation selects a local utility maximizer and drops the probability axis by its maximum, as in the source. Global constancy is derived structurally in Semantics.lean. The elementary cancellation lemma below gives bucket constancy on nonzero-outside contexts; Guard/Complete.lean now extends the exact diagnostic to every row, including zero outside mass. Ties in argmax use a fixed classical choice, not the Julia first-label rule.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

noncomputable section

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

variable {bn : FinBayesNet}

def Depends (S : Finset bn.V) (f : bn.Assignment → ℝ) : Prop :=
  ∀ x y, (∀ v ∈ S, x v = y v) → f x = f y

structure Valuation (bn : FinBayesNet) where

```

```

scope : Finset bn.V
prob : bn.Assignment → R
util : bn.Assignment → R
prob_local : Depends scope prob
util_local : Depends scope util
nonneg : ∀ x, 0 ≤ prob x

namespace Valuation

def weight (v : Valuation bn) (x : bn.Assignment) : R := v.prob x * v.util x

def unit : Valuation bn where
  scope := ∅
  prob := fun _ => 1
  util := fun _ => 0
  prob_local := fun _ _ => rfl
  util_local := fun _ _ => rfl
  nonneg := fun _ => zero_le_one

def combine (v w : Valuation bn) : Valuation bn where
  scope := v.scope ∪ w.scope
  prob x := v.prob x * w.prob x
  util x := v.util x + w.util x
  prob_local x y h := by
    dsimp only
    rw [v.prob_local x y (fun a ha => h a (Finset.mem_union_left _ ha)),
        w.prob_local x y (fun a ha => h a (Finset.mem_union_right _ ha))]
  util_local x y h := by
    dsimp only
    rw [v.util_local x y (fun a ha => h a (Finset.mem_union_left _ ha)),
        w.util_local x y (fun a ha => h a (Finset.mem_union_right _ ha))]
  nonneg x := mul_nonneg (v.nonneg x) (w.nonneg x)

def collect : List (Valuation bn) → Valuation bn
| [] => unit
| v :: vs => combine v (collect vs)

def bucket (a : bn.V) (vs : List (Valuation bn)) : List (Valuation bn) :=
  vs.filter (fun v => decide (a ∈ v.scope))

def outside (a : bn.V) (vs : List (Valuation bn)) : List (Valuation bn) :=
  vs.filter (fun v => decide (a ∉ v.scope))

theorem collect_scope (vs : List (Valuation bn)) {v : Valuation bn} (hv : v ∈ vs) :
  v.scope ⊆ (collect vs).scope := by
  induction vs with
  | nil => simp at hv
  | cons w ws ih =>
    rcases List.mem_cons.1 hv with rfl | hv
    • exact Finset.subset_union_left
    • exact (ih hv).trans Finset.subset_union_right

theorem outside_notMem (a : bn.V) (vs : List (Valuation bn)) :
  a ∉ (collect (outside a vs)).scope := by
  induction vs with
  | nil => simp [outside, collect, unit]
  | cons v vs ih =>
    by_cases h : a ∈ v.scope <|> simp [outside, collect, combine, h] using ih

theorem collect_partition (a : bn.V) (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect vs).prob x =
    (collect (bucket a vs)).prob x * (collect (outside a vs)).prob x ∧
  (collect vs).util x =
    (collect (bucket a vs)).util x + (collect (outside a vs)).util x := by
  induction vs with
  | nil => simp [bucket, outside, collect, unit]

```

```

| cons v vs ih =>
  by_cases h : a ∈ v.scope <=>
    simp [bucket, outside, collect, combine, h] at * <=>
    rcases ih with (hp, hu) <=> rw [hp, hu] <=> constructor <=> ring

theorem collect_partition_scope (a : bn.V) (vs : List (Valuation bn)) :
  (collect vs).scope =
    (collect (bucket a vs)).scope ∪ (collect (outside a vs)).scope := by
  induction vs with
  | nil => simp [bucket, outside, collect, unit]
  | cons v vs ih =>
    by_cases h : a ∈ v.scope <=>
      simp [bucket, outside, collect, combine, h] at * <=> (rw [ih]; try ac_rfl)

theorem prob_update_of_notMem (v : Valuation bn) {a : bn.V} (ha : a ∉ v.scope)
  (x : bn.Assignment) (b : bn.states a) : v.prob (Function.update x a b) = v.prob x := by
  apply v.prob_local
  intro w hw
  exact Function.update_of_ne (show w ≠ a from fun he => ha (he ▸ hw)) _ _

theorem util_update_of_notMem (v : Valuation bn) {a : bn.V} (ha : a ∉ v.scope)
  (x : bn.Assignment) (b : bn.states a) : v.util (Function.update x a b) = v.util x := by
  apply v.util_local
  intro w hw
  exact Function.update_of_ne (show w ≠ a from fun he => ha (he ▸ hw)) _ _

theorem update_agree {S : Finset bn.V} {a : bn.V} {x y : bn.Assignment}
  (h : ∀ v ∈ S.erase a, x v = y v) (b : bn.states a) :
  ∀ v ∈ S, Function.update x a b v = Function.update y a b v := by
  intro v hv
  by_cases he : v = a
  · subst v
    simp
  · simp only [Function.update_of_ne he]
    exact h v (Finset.mem_erase.2 (he, hv))

def ratio (n p : ℝ) : ℝ := if p = 0 then 0 else n / p

theorem mass_mul_ratio {n p : ℝ} (h : p = 0 → n = 0) : p * ratio n p = n := by
  by_cases hp : p = 0
  · simp [ratio, hp, h hp]
  · simp [ratio, hp, mul_div_cancel₀]

theorem weighted_sum_zero {A : Type} [Fintype A] (p u : A → ℝ)
  (hp : ∀ a, 0 ≤ p a) (hz : ∑ a, p a = 0) : ∑ a, p a * u a = 0 := by
  have h : ∀ a, p a = 0 := by
    intro a
    exact (Finset.sum_eq_zero_iff_of_nonneg (fun a _ => hp a)).1 hz a (Finset.mem_univ _)
  simp [h]

def sumOut (a : bn.V) (v : Valuation bn) : Valuation bn where
  scope := v.scope.erase a
  prob x := ∑ b, v.prob (Function.update x a b)
  util x := ratio (∑ b, v.weight (Function.update x a b))
    (∑ b, v.prob (Function.update x a b))
  prob_local x y h := Finset.sum_congr rfl fun b _ => v.prob_local _ _ (update_agree h b)
  util_local x y h := by
    dsimp only
    apply congrArg2 ratio <=> apply Finset.sum_congr rfl <=> intro b _
    · simp only [weight, v.prob_local _ _ (update_agree h b),
      v.util_local _ _ (update_agree h b)]
    · exact v.prob_local _ _ (update_agree h b)
  nonneg x := Finset.sum_nonneg fun b _ => v.nonneg _

/-- This includes zero-probability rows; no division by a positive number is assumed. -/
theorem sumOut_weight (a : bn.V) (v : Valuation bn) (x : bn.Assignment) :

```

```

    (sumOut a v).weight x =  $\sum$  b, v.weight (Function.update x a b) := by
  apply mass_mul_ratio
  exact weighted_sum_zero _ _ (fun b  $\Rightarrow$  v.nonneg _)

noncomputable def argmax {A : Type} [Fintype A] [Nonempty A] (f : A  $\rightarrow$   $\mathbb{R}$ ) : A :=
  Classical.choose (Finset.exists_max_image Finset.univ f Finset.univ_nonempty)

theorem le_argmax {A : Type} [Fintype A] [Nonempty A] (f : A  $\rightarrow$   $\mathbb{R}$ ) (a : A) :
  f a  $\leq$  f (argmax f) :=
  (Classical.choose_spec (Finset.exists_max_image Finset.univ f Finset.univ_nonempty)).2 a
  (Finset.mem_univ a)

noncomputable def choice (a : bn.V) (v : Valuation bn) (x : bn.Assignment) : bn.states a :=
  argmax fun b  $\Rightarrow$  v.util (Function.update x a b)

theorem choice_local (a : bn.V) (v : Valuation bn) (x y : bn.Assignment)
  (h :  $\forall w \in v.scope.erase a, x w = y w$ ) : choice a v x = choice a v y := by
  apply congrArg argmax
  funext b
  exact v.util_local _ _ (update_agree h b)

def probabilityChoice (a : bn.V) (v : Valuation bn) (x : bn.Assignment) : bn.states a :=
  argmax fun b  $\Rightarrow$  v.prob (Function.update x a b)

theorem probabilityChoice_local (a : bn.V) (v : Valuation bn) (x y : bn.Assignment)
  (h :  $\forall w \in v.scope.erase a, x w = y w$ ) :
  probabilityChoice a v x = probabilityChoice a v y := by
  apply congrArg argmax
  funext b
  exact v.prob_local _ _ (update_agree h b)

noncomputable def maxOut (a : bn.V) (v : Valuation bn) : Valuation bn where
  scope := v.scope.erase a
  prob x := v.prob (Function.update x a (probabilityChoice a v x))
  util x := v.util (Function.update x a (choice a v x))
  prob_local x y h := by
    dsimp only
    rw [probabilityChoice_local a v x y h]
    exact v.prob_local _ _ (update_agree h _)
  util_local x y h := by
    dsimp only
    rw [choice_local a v x y h]
    exact v.util_local _ _ (update_agree h _)
  nonneg x := v.nonneg _

def chanceStep (a : bn.V) (vs : List (Valuation bn)) : List (Valuation bn) :=
  sumOut a (collect (bucket a vs)) :: outside a vs

noncomputable def decisionStep (a : bn.V) (vs : List (Valuation bn)) : List (Valuation bn) :=
  maxOut a (collect (bucket a vs)) :: outside a vs

theorem chanceStep_prob (a : bn.V) (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect (chanceStep a vs)).prob x =  $\sum$  b, (collect vs).prob (Function.update x a b) := by
  change ( $\sum$  b, (collect (bucket a vs)).prob (Function.update x a b)) *
  (collect (outside a vs)).prob x = _
  rw [Finset.sum_mul]
  refine Finset.sum_congr rfl fun b _  $\Rightarrow$  ?_
  rw [(collect_partition a vs _).1, prob_update_of_notMem _ (outside_notMem a vs)]

/-- Full bucket identity, including the utility contribution of untouched valuations. -/
theorem chanceStep_weight (a : bn.V) (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect (chanceStep a vs)).weight x =
   $\sum$  b, (collect vs).weight (Function.update x a b) := by
  let v := collect (bucket a vs)
  let r := collect (outside a vs)
  have hw := sumOut_weight a v x

```

```

change (sumOut a v).prob x * (sumOut a v).util x = _ at hw
change ((sumOut a v).prob x * r.prob x) * ((sumOut a v).util x + r.util x) = _
calc
- = r.prob x * (( $\sum$  b, v.weight (Function.update x a b)) +
  ( $\sum$  b, v.prob (Function.update x a b)) * r.util x) := by
  rw [← hw]
  change (sumOut a v).prob x * r.prob x * ((sumOut a v).util x + r.util x) =
    r.prob x * ((sumOut a v).prob x * (sumOut a v).util x +
      (sumOut a v).prob x * r.util x)
  ring
- = _ := by
  rw [Finset.sum_mul, ← Finset.sum_add_distrib, Finset.mul_sum]
  refine Finset.sum_congr rfl fun b _ => ?_
  simp only [weight]
  rw [(collect_partition a vs _).1, (collect_partition a vs _).2,
    prob_update_of_notMem _ (outside_notMem a vs),
    util_update_of_notMem _ (outside_notMem a vs)]
  dsimp [v, r, weight]
  ring

theorem decisionStep_eval (a : bn.V) (vs : List (Valuation bn))
  (hp :  $\forall$  x b, (collect vs).prob (Function.update x a b) = (collect vs).prob x)
  (x : bn.Assignment) :
  let y := Function.update x a (choice a (collect (bucket a vs)) x)
  (collect (decisionStep a vs)).prob x = (collect vs).prob y  $\wedge$ 
    (collect (decisionStep a vs)).util x = (collect vs).util y := by
  dsimp only
  constructor
  · have he : (collect (decisionStep a vs)).prob x =
    (collect vs).prob (Function.update x a (probabilityChoice a (collect (bucket a vs)) x)) := by
    rw [(collect_partition a vs _).1, prob_update_of_notMem _ (outside_notMem a vs)]
    rfl
    rw [he, hp, hp]
  · rw [(collect_partition a vs _).2, util_update_of_notMem _ (outside_notMem a vs)]
    rfl

/-- The literal bucket probability guard is justified on every nonzero-outside context.
At zero outside mass, the whole configuration is unreachable and cancellation is invalid. -/
theorem bucket_probability_constant_on_support (a : bn.V) (vs : List (Valuation bn))
  (hp :  $\forall$  x b, (collect vs).prob (Function.update x a b) = (collect vs).prob x)
  (x : bn.Assignment) (hout : (collect (outside a vs)).prob x  $\neq$  0) (b : bn.states a) :
  (collect (bucket a vs)).prob (Function.update x a b) = (collect (bucket a vs)).prob x := by
  apply mul_right_cancel0 hout
  have h := hp x b
  rw [(collect_partition a vs _).1, (collect_partition a vs _).1,
    prob_update_of_notMem _ (outside_notMem a vs)] at h
  exact h

/-- The global probability-independence obligation is explicit and not assumed by the driver.
Later modules derive it from the causal order and normalisation. -/
theorem decisionStep_dominates (a : bn.V) (vs : List (Valuation bn))
  (hp :  $\forall$  x b, (collect vs).prob (Function.update x a b) = (collect vs).prob x)
  (x : bn.Assignment) (b : bn.states a) :
  (collect vs).weight (Function.update x a b)  $\leq$  (collect (decisionStep a vs)).weight x := by
  rw [weight, weight, (decisionStep_eval a vs hp x).1, (decisionStep_eval a vs hp x).2, hp, hp]
  apply mul_le_mul_of_nonneg_left _ ((collect vs).nonneg x)
  rw [(collect_partition a vs _).2, (collect_partition a vs _).2,
    util_update_of_notMem _ (outside_notMem a vs),
    util_update_of_notMem _ (outside_notMem a vs)]
  exact add_le_add (le_argmax (fun b => (collect (bucket a vs)).util
    (Function.update x a b)) b) (le_refl _)

theorem filter_scope_subset (p : Valuation bn  $\rightarrow$  Bool) (vs : List (Valuation bn)) :
  (collect (vs.filter p)).scope  $\subseteq$  (collect vs).scope := by
  induction vs with
  | nil => exact Finset.Subset.refl _

```

```

| cons v vs ih =>
  by_cases h : p v
  · simp only [List.filter_cons, h, ↓reduceIte, collect, combine]
    exact Finset.union_subset_union (Finset.Subset.refl _) ih
  · simp only [List.filter_cons, h, Bool.false_eq_true, ↓reduceIte, collect, combine]
    exact ih.trans Finset.subset_union_right

theorem step_scope (a : bn.V) (vs : List (Valuation bn))
  (reduce : Valuation bn → Valuation bn) (hr : ∀ v, (reduce v).scope = v.scope.erase a) :
  (collect (reduce (collect (bucket a vs))) :: outside a vs).scope ⊆
  (collect vs).scope.erase a := by
change (reduce _).scope ∪ (collect (outside a vs)).scope ⊆ _
rw [hr]
apply Finset.union_subset
· exact Finset.erase_subset_erase a (filter_scope_subset _ vs)
· intro w hw
  refine Finset.mem_erase.2 {?, filter_scope_subset _ vs hw}
  exact fun he => outside_notMem a vs (he ▸ hw)

theorem step_scope_eq (a : bn.V) (vs : List (Valuation bn))
  (reduce : Valuation bn → Valuation bn) (hr : ∀ v, (reduce v).scope = v.scope.erase a) :
  (collect (reduce (collect (bucket a vs))) :: outside a vs).scope =
  (collect vs).scope.erase a := by
change (reduce _).scope ∪ (collect (outside a vs)).scope = _
rw [hr, collect_partition_scope a vs]
ext w
by_cases hw : w = a
· subst w
  simp [outside_notMem a vs]
· simp [hw]

end Valuation
end
end InfluenceDiagramsProofs.DVE

```

Structural probability independence

```
import InfluenceDiagramsProofs.Finite.DVE.Valuation
```

Uneliminated decisions are free inputs: their factors are one, not probability distributions. Eliminated decisions carry the reconstructed, local normalised policies. The decisive theorem below derives action-independence of a remaining probability marginal from acyclicity, generative uniqueness, normalisation and the strong-order information boundary. It does not take a semantic legality certificate or solver/oracle equality as a premise.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram

noncomputable section

variable {id : FinInfluenceDiagram}

/-- Numeric topological ranks for the causal and information arcs. -/
structure RankedOrder (id : FinInfluenceDiagram) where
  order : id.IDOrder
  rank : id.V → ℕ
  parents_lt : ∀ m v, v ∈ id.parents m → rank v < rank (id.target m)
  info_lt : ∀ d v, v ∈ id.info d → rank v < rank (id.action d)

def freeJoint (κ : id.Kernel ℝ) (E : Finset id.V) (σ : Strategy id ℝ)
  (x : id.Assignment) : ℝ :=

```

```

( $\prod m, \kappa m \times (x (id.target m))$ ) *
 $\prod d, \text{if } id.action d \in E \text{ then } (\sigma d).kernel \times (x (id.action d)) \text{ else } 1$ 

def activeKernel ( $\kappa : id.Kernel \mathbb{R}$ ) ( $E : Finset id.V$ ) ( $\sigma : Strategy id \mathbb{R}$ ) :
  id.instantiate.Kernel  $\mathbb{R}$ 
| .inl m  $\Rightarrow \kappa m$ 
| .inr d  $\Rightarrow \text{if } id.action d \in E \text{ then } (\sigma d).kernel \text{ else fun } \_ \Rightarrow 1$ 

theorem joint_active ( $\kappa : id.Kernel \mathbb{R}$ ) ( $E : Finset id.V$ ) ( $\sigma : Strategy id \mathbb{R}$ ) :
  joint (activeKernel  $\kappa E \sigma$ ) = freeJoint  $\kappa E \sigma := \text{by}$ 
  funext x
  unfold joint freeJoint
  rw [Fintype.prod_sum_type]
  congr 1
  apply Finset.prod_congr rfl
  intro d _
  by_cases h : id.action d  $\in E$  <=> simp [activeKernel, h, instantiate]

theorem active_local ( $\kappa : id.Kernel \mathbb{R}$ ) ( $h\kappa : \forall m, Local \kappa m$ )
  ( $E : Finset id.V$ ) ( $\sigma : Strategy id \mathbb{R}$ ) :  $\forall m, Local (activeKernel \kappa E \sigma) m := \text{by}$ 
  rintro (m | d)
  · exact h $\kappa m$ 
  · by_cases h : id.action d  $\in E$ 
    · exact (show Local (activeKernel  $\kappa E \sigma$ ) (.inr d) from by
      simp [Local, activeKernel, h, instantiate] using ( $\sigma d$ ).localOn)
    · intro x y _
      simp [activeKernel, h]

theorem freeJoint_empty ( $\kappa : id.Kernel \mathbb{R}$ ) ( $\sigma : Strategy id \mathbb{R}$ ) ( $x : id.Assignment$ ) :
  freeJoint  $\kappa \emptyset \sigma x = \prod m, \kappa m \times (x (id.target m)) := \text{by simp [freeJoint]}$ 

theorem freeJoint_univ ( $\kappa : id.Kernel \mathbb{R}$ ) ( $\sigma : Strategy id \mathbb{R}$ ) :
  freeJoint  $\kappa Finset.univ \sigma = joint (strategyKernel \kappa \sigma) := \text{by}$ 
  funext x
  rw [joint_instantiate]
  simp [freeJoint]

theorem freeJoint_insert_chance ( $\kappa : id.Kernel \mathbb{R}$ ) ( $E : Finset id.V$ ) ( $\sigma : Strategy id \mathbb{R}$ )
  ( $v : id.V$ ) ( $h_v : \forall d, id.action d \neq v$ ) :
  freeJoint  $\kappa (insert v E) \sigma = freeJoint \kappa E \sigma := \text{by}$ 
  funext x
  simp [freeJoint, h_v]

theorem freeJoint_activate ( $\kappa : id.Kernel \mathbb{R}$ ) ( $E : Finset id.V$ ) ( $\sigma : Strategy id \mathbb{R}$ )
  ( $h_{inj} : Function.Injective id.action$ ) ( $d : id.D$ ) ( $hd : id.action d \notin E$ )
  ( $\pi : Policy id \mathbb{R} d$ ) ( $x : id.Assignment$ ) :
  freeJoint  $\kappa (insert (id.action d) E) (Function.update \sigma d \pi) x =$ 
  freeJoint  $\kappa E \sigma x * \pi.kernel \times (x (id.action d)) := \text{by}$ 
  have hfac :  $\forall e,$ 
    (if id.action e  $\in insert (id.action d) E$  then
      (Function.update  $\sigma d \pi e$ ).kernel  $\times (x (id.action e))$  else 1) =
    (if id.action e  $\in E$  then  $(\sigma e).kernel \times (x (id.action e))$  else 1) *
    (if e = d then  $\pi.kernel \times (x (id.action d))$  else 1) := by
  intro e
  by_cases he : e = d
  · subst e
    simp [hd]
  · have ha : id.action e  $\neq id.action d := \text{fun } h \Rightarrow he (h_{inj} h)$ 
    simp [he, ha]
  unfold freeJoint
  simp_rw [hfac]
  rw [Finset.prod_mul_distrib]
  simp [mul_assoc]

section Marginals

```

```

variable {bn : FinBayesNet}

theorem marg_mono {S : Finset bn.V} {f g : bn.Assignment → ℝ}
  (h : ∀ x, f x ≤ g x) (x : bn.Assignment) : marg S f x ≤ marg S g x :=
  Finset.sum_le_sum fun y _ => h y

theorem marg_preserves_independence (S : Finset bn.V) (v : bn.V) (hv : v ∉ S)
  (F : bn.Assignment → ℝ) (hF : ∀ x a, F (Function.update x v a) = F x)
  (x : bn.Assignment) (a : bn.states v) :
  marg S F (Function.update x v a) = marg S F x := by
induction S using Finset.induction_on generalizing x with
| empty => simp [marg_empty] using hF x a
| @insert w S hw ih =>
  have hvw : v ≠ w := fun h => hv (Finset.mem_insert.2 (Or.inl h))
  have hvS : v ∉ S := fun h => hv (Finset.mem_insert_of_mem h)
  rw [marg_insert hw, marg_insert hw]
  refine Finset.sum_congr rfl fun b _ => ?_
  rw [Function.update_comm hvw, ih hvS]

theorem marg_policy_factor (E : Finset id.V) (d : id.D) (hd : id.action d ∉ E)
  (hinfo : ∀ v ∈ id.info d, v ∉ insert (id.action d) E)
  (π : Policy id ℝ d) (F : id.Assignment → ℝ) (x : id.Assignment) :
  marg (insert (id.action d) E) (fun y => F y * π.kernel y (y (id.action d))) x =
  ∑ a, π.kernel x a * marg E F (Function.update x (id.action d) a) := by
rw [marg_insert hd]
refine Finset.sum_congr rfl fun a _ => ?_
unfold marg
rw [Finset.mul_sum]
refine Finset.sum_congr rfl fun y hy => ?_
have hya : y (id.action d) = a := by
  simp using mem_fibre.1 hy (id.action d) hd
have hπ : π.kernel y = π.kernel x := by
  apply π.localOn
  intro v hv
  have hnot := hinfo v hv
  have hne : v ≠ id.action d := fun he => hnot (Finset.mem_insert.2 (Or.inl he))
  rw [mem_fibre.1 hy v (fun h => hnot (Finset.mem_insert_of_mem h)),
    Function.update_of_ne hne]
dsimp only
rw [hπ, hya, mul_comm]

theorem decision_marginal (κ : id.Kernel ℝ) (E : Finset id.V) (σ : Strategy id ℝ)
  (hinj : Function.Injective id.action) (d : id.D) (hd : id.action d ∉ E)
  (hinfo : ∀ v ∈ id.info d, v ∉ insert (id.action d) E) (π : Policy id ℝ d)
  (F : id.Assignment → ℝ) (x : id.Assignment) :
  marg (insert (id.action d) E)
    (fun y => freeJoint κ (insert (id.action d) E) (Function.update σ d π) y * F y) x =
  ∑ a, π.kernel x a * marg E (fun y => freeJoint κ E σ y * F y)
    (Function.update x (id.action d) a) := by
have heq : (fun y => freeJoint κ (insert (id.action d) E) (Function.update σ d π) y * F y) =
  (fun y => (freeJoint κ E σ y * F y) * π.kernel y (y (id.action d))) := by
  funext y
  rw [freeJoint_activate κ E σ hinj d hd π]
  ring
rw [heq]
exact marg_policy_factor E d hd hinfo π _ x

theorem decision_marginal_pure (κ : id.Kernel ℝ) (E : Finset id.V) (σ : Strategy id ℝ)
  (hinj : Function.Injective id.action) (d : id.D) (hd : id.action d ∉ E)
  (hinfo : ∀ v ∈ id.info d, v ∉ insert (id.action d) E)
  (f : id.Assignment → id.states (id.action d))
  (hf : ∀ x y, (∀ v ∈ id.info d, x v = y v) → f x = f y)
  (F : id.Assignment → ℝ) (x : id.Assignment) :
  marg (insert (id.action d) E)
    (fun y => freeJoint κ (insert (id.action d) E)
      (Function.update σ d (Policy.ofFun f hf)) y * F y) x =

```

```

    marg E (fun y ⇒ freeJoint κ E σ y * F y) (Function.update x (id.action d) (f x)) := by
  rw [decision_marginal κ E σ hnj d hd hinfo]
  simp [Policy.ofFun_kernel, ite_mul]

end Marginals

/-- Under a strong information boundary, the probability marginal cannot depend on the
current free action. Nonempty state spaces and all policy normalisations come from the model
and 'Policy'; chance normalisation and locality are explicit. Zero masses are allowed. -/
theorem probability_independent_weighted (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : RankedOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (E : Finset id.V) (σ : Strategy id R) (d : id.D) (hd : id.action d ∉ E)
  (hboundary : Ec ⊆ insert (id.action d) (id.info d))
  (G : id.Assignment → R)
  (hG : Depends (Finset.univ.filter fun v ⇒ ord.rank v ≤ ord.rank (id.action d)) G)
  (hGv : ∀ y b, G (Function.update y (id.action d) b) = G y)
  (x : id.Assignment) (a : id.states (id.action d)) :
  marg E (fun y ⇒ freeJoint κ E σ y * G y) (Function.update x (id.action d) a) =
  marg E (fun y ⇒ freeJoint κ E σ y * G y) x := by
  let U : Finset id.V := Finset.univ.filter fun v ⇒ ord.rank v ≤ ord.rank (id.action d)
  have hRU : Ec ⊆ U := by
    intro v hv
    rcases Finset.mem_insert.1 (hboundary hv) with rfl | hi
    · simp [U]
    · exact Finset.mem_filter.2 ⟨Finset.mem_univ _, (ord.info_lt d v hi).le⟩
  have hUE : Uc ⊆ E := by
    intro v hv
    by_contra he
    exact Finset.mem_compl.1 hv (hRU (Finset.mem_compl.2 he))
  have hU : UpstreamClosed (bn := id.instantiate) U := by
    rintro (m | e) hm v hv
    · exact Finset.mem_filter.2 ⟨Finset.mem_univ _,
      (ord.parents_lt m v hv).le.trans (Finset.mem_filter.1 hm).2⟩
    · exact Finset.mem_filter.2 ⟨Finset.mem_univ _,
      (ord.info_lt e v hv).le.trans (Finset.mem_filter.1 hm).2⟩
  have hn : ∀ m, id.instantiate.target m ∉ U → Normalised (activeKernel κ E σ) m := by
    rintro (m | e) hm
    · exact hnorm m
    · have he : id.action e ∈ E := hUE (Finset.mem_compl.2 hm)
      simpa [Normalised, activeKernel, he] using (σ e).normalised
  let F : id.Assignment → R := fun y ⇒
    ∏ m ∈ Finset.univ.filter (fun m ⇒ id.instantiate.target m ∈ U),
    activeKernel κ E σ m y (y (id.instantiate.target m))
  have hdown : ∀ y, marg Uc (freeJoint κ E σ) y = F y := by
    intro y
    rw [← joint_active]
    exact marg_joint_downstream _ (closed_instantiate hclosed).1 ord.order.toTopoOrder
      (active_local κ hloc E σ) U hn hU (fun v _ ⇒ (closed_instantiate hclosed).2 v) y
  have hF : ∀ y b, F (Function.update y (id.action d) b) = F y := by
    intro y b
    refine Finset.prod_congr rfl fun g hg ⇒ ?_
    have hgU := (Finset.mem_filter.1 hg).2
    cases g with
    | inl m ⇒
      have hne : id.target m ≠ id.action d := (id.closed_iff.1 hclosed).2.2.1 m d
      have hparent : id.action d ∉ id.parents m := by
        intro h
        exact (Nat.not_lt_of_ge (Finset.mem_filter.1 hgU).2) (ord.parents_lt m _ h)
      change κ m (Function.update y (id.action d) b)
      (Function.update y (id.action d) b (id.target m)) = _
      rw [Function.update_of_ne hne, hloc m _ y (fun v hv ⇒
        Function.update_of_ne (show v ≠ id.action d from fun he ⇒ hparent (he ▶ hv)) _ _)]
      rfl
    | inr e ⇒
      by_cases he : id.action e = id.action d
      · have hed : e = d := (id.closed_iff.1 hclosed).2.1 he

```

```

    subst e
    simp [activeKernel, hd]
    • have hinfo : id.action d ∉ id.info e := by
      intro h
      exact (Nat.not_lt_of_ge (Finset.mem_filter.1 hgU).2) (ord.info_lt e _ h)
    by_cases hE : id.action e ∈ E
    • simp only [activeKernel, hE, ↓reduceIte]
      change (σ e).kernel (Function.update y (id.action d) b)
        (Function.update y (id.action d) b (id.action e)) =
        (σ e).kernel y (y (id.action e))
      rw [Function.update_of_ne he, (σ e).localOn _ y (fun v hv =>
        Function.update_of_ne (show v ≠ id.action d from fun hv => hinfo (he ▸ hv)) _ _)]
    • simp [activeKernel, hE]
  have hset : E = (E ∩ U) ∪ Uc := by
    ext v
    by_cases hu : v ∈ U
    • simp [hu]
    • have he := hUE (Finset.mem_compl.2 hu)
      simp [hu, he]
  have hdisj : Disjoint (E ∩ U) Uc :=
    Finset.disjoint_left.2 fun _ hi hc => Finset.mem_compl.1 hc (Finset.mem_inter.1 hi).2
  have hweighted : ∀ y, marg Uc (fun z => freeJoint κ E σ z * G z) y = F y * G y := by
    intro y
    calc
    _ = marg Uc (fun z => G z * freeJoint κ E σ z) y := by
      apply congrArg (fun f => marg Uc f y)
      funext z
      ring
    _ = G y * marg Uc (freeJoint κ E σ) y :=
      marg_mul_left (fun z hz => hG z y fun v hv =>
        mem_fibre.1 hz v (fun hc => Finset.mem_compl.1 hc hv))
    _ = _ := by rw [hdown]; ring
  have hm : ∀ y, marg E (fun z => freeJoint κ E σ z * G z) y =
    marg (E ∩ U) (fun z => F z * G z) y := by
    intro y
    calc
    _ = marg ((E ∩ U) ∪ Uc) (fun z => freeJoint κ E σ z * G z) y :=
      congrArg (fun S => marg S (fun z => freeJoint κ E σ z * G z) y) hset
    _ = marg (E ∩ U) (fun z => marg Uc (fun w => freeJoint κ E σ w * G w) z) y :=
      marg_union_disjoint hdisj _ _
    _ = _ := congrArg (fun f => marg (E ∩ U) f y) (funext hweighted)
  rw [hm, hm]
  apply marg_preserves_independence _ _ (fun h => hd (Finset.mem_inter.1 h).1) _ _ x a
  intro y b
  rw [hF, hGv]

/-- The no-evidence specialization of structural probability independence. -/
theorem probability_independent (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : RankedOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (E : Finset id.V) (σ : Strategy id R) (d : id.D) (hd : id.action d ∉ E)
  (hboundary : Ec ⊆ insert (id.action d) (id.info d))
  (x : id.Assignment) (a : id.states (id.action d)) :
  marg E (freeJoint κ E σ) (Function.update x (id.action d) a) =
  marg E (freeJoint κ E σ) x := by
  simp using probability_independent_weighted κ hclosed ord hloc hnorm E σ d hd hboundary
  (fun _ => 1) (fun _ _ => rfl) (fun _ _ => rfl) x a
end
end InfluenceDiagramsProofs.DVE

```

Bucket driver and policy reconstruction

```
import InfluenceDiagramsProofs.Finite.DVE.Semantics
```

Plan contains only variable identities and information-set equalities. In particular it contains no probability independence, value bound, solver correctness, or oracle premise. `run` executes bucket chance elimination and local utility maximisation, recording each chosen policy. The semantic invariant is proved by induction; decision probability independence is obtained from the structural theorem in `Semantics`. `Lean`, not from the plan.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram
open Valuation

noncomputable section

variable {id : FinInfluenceDiagram}

inductive Plan (id : FinInfluenceDiagram) : Finset id.V → Type
| done : Plan id ∅
| chance {R} (v : id.V) (hv : v ∈ R) (chance : ∀ d, id.action d ≠ v)
  (next : Plan id (R.erase v)) : Plan id R
| decision {R} (d : id.D) (hd : id.action d ∈ R)
  (information : R.erase (id.action d) = id.info d)
  (next : Plan id (R.erase (id.action d))) : Plan id R

structure State (id : FinInfluenceDiagram) (R : Finset id.V) where
  valuations : List (Valuation id.toFinBayesNet)
  supported : (collect valuations).scope ⊆ R

namespace State

variable {R : Finset id.V}

def chance (v : id.V) (s : State id R) : State id (R.erase v) where
  valuations := chanceStep v s.valuations
  supported := (step_scope v s.valuations (sumOut v) (fun _ => rfl)).trans
    (Finset.erase_subset_erase v s.supported)

def decision (d : id.D) (s : State id R) : State id (R.erase (id.action d)) where
  valuations := decisionStep (id.action d) s.valuations
  supported := (step_scope (id.action d) s.valuations (maxOut (id.action d)) (fun _ => rfl)).trans
    (Finset.erase_subset_erase _ s.supported)

def policy (d : id.D) (s : State id R) (hi : R.erase (id.action d) = id.info d) :
  Policy id R d :=
  Policy.ofFun (choice (id.action d) (collect (bucket (id.action d) s.valuations))) (by
    intro x y h
    apply choice_local
    intro v hv
    apply h
    rw [← hi]
    exact Finset.erase_subset_erase _ ((filter_scope_subset _ s.valuations).trans s.supported) hv)

end State

/-- The actual solver: local bucket updates, not enumeration or argmax over strategies. -/
def run {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R) :
  State id ∅ × Strategy id R :=
  match plan with
  | .done => (s, σ)
  | .chance v _ _ next => run next (s.chance v) σ
  | .decision d _ hi next => run next (s.decision d) (Function.update σ d (s.policy d hi))

/-- Exact mass and payoff realization, together with the upper bound for every competitor.
This is an invariant to prove, not data required by 'Plan' or 'run'. -/
structure Correct (κ : id.Kernel R) (u : Utility id R) {R : Finset id.V}
  (s : State id R) (σ : Strategy id R) : Prop where

```

```

mass : ∀ x, (collect s.valuations).prob x = marg Rc (freeJoint κ Rc σ) x
realizes : ∀ x, (collect s.valuations).weight x =
  marg Rc (fun y ⇒ freeJoint κ Rc σ y * totalUtility u y) x
dominates : ∀ τ : Strategy id R, τ.Nonneg → ∀ x,
  marg Rc (fun y ⇒ freeJoint κ Rc τ y * totalUtility u y) x ≤
    (collect s.valuations).weight x

theorem compl_erase (R : Finset id.V) (v : id.V) :
  (R.erase v)c = insert v Rc := by
  ext w
  by_cases h : w = v <|> simp [h]

theorem Correct.chance {κ : id.Kernel R} {u : Utility id R} {R : Finset id.V}
  {s : State id R} {σ : Strategy id R} (h : Correct κ u s σ)
  (v : id.V) (hv : v ∈ R) (hc : ∀ d, id.action d ≠ v) :
  Correct κ u (s.chance v) σ := by
  have hn : v ∉ Rc := by simp using hv
  constructor
  · intro x
    change (collect (chanceStep v s.valuations)).prob x = _
    rw [chanceStep_prob, compl_erase, freeJoint_insert_chance κ Rc σ v hc, marg_insert hn]
    exact Finset.sum_congr rfl fun b _ => h.mass _
  · intro x
    change (collect (chanceStep v s.valuations)).weight x = _
    rw [chanceStep_weight, compl_erase, freeJoint_insert_chance κ Rc σ v hc, marg_insert hn]
    exact Finset.sum_congr rfl fun b _ => h.realizes _
  · intro τ ht x
    change _ ≤ (collect (chanceStep v s.valuations)).weight x
    rw [chanceStep_weight, compl_erase, freeJoint_insert_chance κ Rc τ v hc, marg_insert hn]
    exact Finset.sum_le_sum fun b _ => h.dominates τ ht _

theorem info_disjoint {R : Finset id.V} (d : id.D)
  (hi : R.erase (id.action d) = id.info d) :
  ∀ v ∈ id.info d, v ∉ insert (id.action d) Rc := by
  intro v hv
  rw [← hi, Finset.mem_erase] at hv
  simp [hv.1, hv.2]

theorem information_boundary {R : Finset id.V} (d : id.D)
  (hi : R.erase (id.action d) = id.info d) :
  R ⊆ insert (id.action d) (id.info d) := by
  intro v hv
  by_cases he : v = id.action d
  · exact Finset.mem_insert.2 (Or.inl he)
  · exact Finset.mem_insert.2 (Or.inr (hi ▸ Finset.mem_erase.2 ⟨he, hv⟩))

theorem Correct.decision {κ : id.Kernel R} {u : Utility id R} {R : Finset id.V}
  {s : State id R} {σ : Strategy id R} (h : Correct κ u s σ)
  (hclosed : id.Closed) (ord : RankedOrder id)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (d : id.D) (hd : id.action d ∈ R) (hi : R.erase (id.action d) = id.info d) :
  Correct κ u (s.decision d) (Function.update σ d (s.policy d hi)) := by
  have hn : id.action d ∉ Rc := by simp using hd
  have hincj := (id.closed_iff.1 hclosed).2.1
  have hinfo := info_disjoint d hi
  let f := choice (id.action d) (collect (bucket (id.action d) s.valuations))
  have hpi : ∀ x a, (s.policy d hi).kernel x a = if a = f x then 1 else 0 := by
    intro x a
    rfl
  have heval : ∀ (F : id.Assignment → R) (x : id.Assignment),
    marg (insert (id.action d) Rc)
      (fun y ⇒ freeJoint κ (insert (id.action d) Rc)
        (Function.update σ d (s.policy d hi)) y * F y) x =
      marg Rc (fun y ⇒ freeJoint κ Rc σ y * F y) (Function.update x (id.action d) (f x)) := by
    intro F x
    rw [decision_marginal κ Rc σ hincj d hn hinfo]

```

```

    simp [hpi, ite_mul]
have hp :  $\forall x a, (\text{collect } s.\text{valuations}).\text{prob } (\text{Function.update } x \text{ (id.action } d) a) =$ 
   $(\text{collect } s.\text{valuations}).\text{prob } x := \text{by}$ 
  intro x a
  rw [h.mass, h.mass]
  exact probability_independent  $\kappa$  hclosed ord hloc hnrm  $R^c \sigma d hn$ 
    (by simp using information_boundary d hi) x a
constructor
• intro x
  change (collect (decisionStep (id.action d) s.valuations)).prob x = _
  rw [(decisionStep_eval _ _ hp x).1, h.mass, compl_erase]
  simp using (heval (fun _  $\Rightarrow$  1) x).symm
• intro x
  change (collect (decisionStep (id.action d) s.valuations)).weight x = _
  rw [weight, (decisionStep_eval _ _ hp x).1, (decisionStep_eval _ _ hp x).2]
  change (collect s.valuations).weight (Function.update x (id.action d) (f x)) = _
  rw [h.realizes, compl_erase, heval]
• intro  $\tau ht x$ 
  change _  $\leq$  (collect (decisionStep (id.action d) s.valuations)).weight x
  rw [compl_erase]
  have hc := decision_marginal  $\kappa R^c \tau hnj d hn hinfo (\tau d) (\text{totalUtility } u) x$ 
  simp only [Function.update_eq_self] at hc
  rw [hc]
  calc
    _  $\leq \sum a, (\tau d).\text{kernel } x a * (\text{collect (decisionStep (id.action d) s.valuations)).weight } x :=$ 
      Finset.sum_le_sum fun a _  $\Rightarrow$  mul_le_mul_of_nonneg_left
        ((h.dominates  $\tau ht$  _).trans (decisionStep_dominates _ _ hp x a)) (ht d x a)
    _ = _ := by rw [ $\leftarrow$  Finset.sum_mul, ( $\tau d$ ).normalised x, one_mul]

/-- The structural/order/normalisation assumptions establish the invariant throughout the
actual recursive bucket driver. In particular, no semantic invariant is assumed by the run. -/
theorem run_correct { $\kappa$  : id.Kernel  $\mathbb{R}$ } {u : Utility id  $\mathbb{R}$ } (hclosed : id.Closed)
  (ord : RankedOrder id) (hloc :  $\forall m, \text{Local } \kappa m$ ) (hnrm :  $\forall m, \text{Normalised } \kappa m$ )
  {R : Finset id.V} (plan : Plan id R) (s : State id R) ( $\sigma$  : Strategy id R)
  (h : Correct  $\kappa u s \sigma$ ) :
  Correct  $\kappa u$  (run plan s  $\sigma$ ).1 (run plan s  $\sigma$ ).2 := by
induction plan generalizing  $\sigma$  with
| done  $\Rightarrow$  exact h
| chance v hv hc next ih  $\Rightarrow$  exact ih (s.chance v)  $\sigma$  (h.chance v hv hc)
| decision d hd hi next ih  $\Rightarrow$ 
  exact ih (s.decision d) _ (h.decision hclosed ord hloc hnrm d hd hi)

end
end InfluenceDiagramsProofs.DVE

```

Initialisation and the returned solution

```
import InfluenceDiagramsProofs.Finite.DVE.Driver
```

Each chance mechanism contributes $(\kappa, 0)$ and each utility contributes $(1, u)$, with its actual dependency scope. The solver starts from those separate valuations, executes run, and returns the scalar utility and reconstructed policies. No strategy enumeration occurs.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section

variable {id : FinInfluenceDiagram}

def chanceValuation ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hloc :  $\forall m, \text{Local } \kappa m$ )

```

```

    (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (m : id.M) : Valuation id.toFinBayesNet where
scope := insert (id.target m) (id.parents m)
prob x :=  $\kappa m \times (x (id.target m))$ 
util _ := 0
prob_local x y h := by
  dsimp only
  rw [hloc m x y (fun v hv  $\Rightarrow$  h v (Finset.mem_insert_of_mem hv)),
    h (id.target m) (Finset.mem_insert_self _ _)]
util_local := fun _ _  $\Rightarrow$  rfl
nonneg x := hnonneg m x _

def utilityValuation (u : Utility id  $\mathbb{R}$ ) (hloc :  $\forall j$ , Utility.Local u j) (j : id.U) :
  Valuation id.toFinBayesNet where
scope := id.uscope j
prob _ := 1
util := u j
prob_local := fun _ _  $\Rightarrow$  rfl
util_local := hloc j
nonneg _ := zero_le_one

def initial ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hloc :  $\forall m$ , Local  $\kappa m$ ) (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ )
  (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j) : State id Finset.univ where
valuations := Finset.univ.toList.map (chanceValuation  $\kappa$  hloc hnonneg) ++
  Finset.univ.toList.map (utilityValuation u hu)
supported := Finset.subset_univ _

theorem collect_prob {bn : FinBayesNet} (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect vs).prob x = (vs.map fun v  $\Rightarrow$  v.prob x).prod := by
induction vs with
| nil  $\Rightarrow$  rfl
| cons v vs ih  $\Rightarrow$  simp [collect, combine, ih]

theorem collect_util {bn : FinBayesNet} (vs : List (Valuation bn)) (x : bn.Assignment) :
  (collect vs).util x = (vs.map fun v  $\Rightarrow$  v.util x).sum := by
induction vs with
| nil  $\Rightarrow$  rfl
| cons v vs ih  $\Rightarrow$  simp [collect, combine, ih]

theorem initial_prob ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hloc :  $\forall m$ , Local  $\kappa m$ )
  (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j)
  (x : id.Assignment) :
  (collect (initial  $\kappa$  hloc hnonneg u hu).valuations).prob x =
     $\prod m, \kappa m \times (x (id.target m))$  := by
simp [initial, collect_prob, List.map_map, chanceValuation, utilityValuation]

theorem initial_util ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hloc :  $\forall m$ , Local  $\kappa m$ )
  (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j)
  (x : id.Assignment) :
  (collect (initial  $\kappa$  hloc hnonneg u hu).valuations).util x = totalUtility u x := by
simp [initial, collect_util, List.map_map, chanceValuation, utilityValuation, totalUtility]

theorem initial_correct ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hloc :  $\forall m$ , Local  $\kappa m$ )
  (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j)
  ( $\sigma$  : Strategy id  $\mathbb{R}$ ) : Correct  $\kappa$  u (initial  $\kappa$  hloc hnonneg u hu)  $\sigma$  := by
constructor
• intro x
  simp [initial_prob, marg_empty, freeJoint_empty]
• intro x
  simp [weight, initial_prob, initial_util, marg_empty, freeJoint_empty]
• intro  $\tau$  _ x
  simp [weight, initial_prob, initial_util, marg_empty, freeJoint_empty]

def CoversChance {R : Finset id.V} (s : State id  $\mathbb{R}$ ) : Prop :=
 $\forall v \in R, (\forall d, id.action d \neq v) \rightarrow v \in (collect s.valuations).scope$ 

theorem initial_covers_chance ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hclosed : id.Closed)

```

```

(hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
(u : Utility id R) (hu : ∀ j, Utility.Local u j) :
CoversChance (initial κ hloc hnonneg u hu) := by
intro v _ hv
obtain ⟨g, hg⟩ := hclosed.2 v
cases g with
| inr d ⇒ exact False.elim (hv d hg)
| inl m ⇒
  have hm : chanceValuation κ hloc hnonneg m ∈ (initial κ hloc hnonneg u hu).valuations := by
    apply List.mem_append_left
    exact List.mem_map.2 ⟨m, Finset.mem_toList.2 (Finset.mem_univ _), rfl⟩
  apply collect_scope _ hm
  change v ∈ insert (id.target m) (id.parents m)
  have he : id.target m = v := hg
  rw [← he]
  exact Finset.mem_insert_self _ _

theorem CoversChance.next {R : Finset id.V} {s : State id R} (h : CoversChance s)
(a : id.V) (t : State id (R.erase a))
(ht : (collect t.valuations).scope = (collect s.valuations).scope.erase a) :
CoversChance t := by
intro v hv hc
rw [ht]
exact Finset.mem_erase.2 ⟨(Finset.mem_erase.1 hv).1, h v (Finset.mem_erase.1 hv).2 hc⟩

def chanceBucketsPresent {R : Finset id.V} (plan : Plan id R) (s : State id R) : Prop :=
match plan with
| .done ⇒ True
| .chance v _ _ next ⇒
  bucket v s.valuations ≠ [] ∧ chanceBucketsPresent next (s.chance v)
| .decision d _ _ next ⇒ chanceBucketsPresent next (s.decision d)

/-- In a closed compiled model, no scheduled chance variable disappears prematurely.
Thus the source's skip-absent-variable optimisation does not change a no-evidence run. -/
theorem chanceBucketsPresent_of_covers {R : Finset id.V} (plan : Plan id R) (s : State id R)
(h : CoversChance s) : chanceBucketsPresent plan s := by
induction plan with
| done ⇒ trivial
| chance v hv hc next ih ⇒
  constructor
  · intro he
    have hs := h v hv hc
    rw [collect_partition_scope v s.valuations, he] at hs
    have hr : v ∈ (collect (outside v s.valuations)).scope := by
      simpa [collect, unit] using hs
    exact outside_notMem v s.valuations hr
  · exact ih _ (h.next v _ (step_scope_eq v s.valuations (sumOut v) (fun _ ⇒ rfl)))
| decision d _ _ next ih ⇒
  exact ih _ (h.next _ _ (step_scope_eq _ s.valuations (maxOut _) (fun _ ⇒ rfl)))

def defaultStrategy : Strategy id R :=
fun d ⇒ Policy.const d (Classical.choice (id.nonemptyS (id.action d)))

theorem defaultStrategy_deterministic : (defaultStrategy (id := id)).Deterministic := by
intro d
exact ⟨fun _ ⇒ Classical.choice (id.nonemptyS (id.action d)), fun _ _ ⇒ rfl, rfl⟩

theorem deterministic_nonneg (σ : Strategy id R) (h : σ.Deterministic) : σ.Nonneg := by
intro d x a
obtain ⟨f, hf, he⟩ := h d
rw [he, Policy.ofFun_kernel]
split_ifs
· exact zero_le_one
· exact le_refl 0

theorem update_policy_deterministic {R : Finset id.V} (s : State id R) (d : id.D)

```

```

(hi : R.erase (id.action d) = id.info d) (σ : Strategy id R) (hσ : σ.Deterministic) :
Strategy.Deterministic (Function.update σ d (s.policy d hi)) := by
intro e
by_cases h : e = d
· subst e
  simp only [Function.update_self]
  unfold State.policy
  refine ⟨_, ?_, rfl⟩
  intro x y h
  apply choice_local
  intro v hv
  apply h
  rw [← hi]
  exact Finset.erase_subset_erase _ ((filter_scope_subset _ s.valuations).trans s.supported) hv
· rw [Function.update_of_ne h]
  exact hσ e

theorem run_deterministic {R : Finset id.V} (plan : Plan id R) (s : State id R)
(σ : Strategy id R) (hσ : σ.Deterministic) : (run plan s σ).2.Deterministic := by
induction plan generalizing σ with
| done ⇒ exact hσ
| chance v _ _ next ih ⇒ exact ih (s.chance v) σ hσ
| decision d _ hi next ih ⇒
  exact ih (s.decision d) _ (update_policy_deterministic s d hi σ hσ)

def baseAssignment (id : FinInfluenceDiagram) : id.Assignment :=
fun v ⇒ Classical.choice (id.nonemptyS v)

structure Solution (id : FinInfluenceDiagram) where
value : R
strategy : Strategy id R

def solvePlan (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
(plan : Plan id Finset.univ) : Solution id :=
let result := run plan (initial κ hloc hnonneg u hu) defaultStrategy
⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩

/-- End-to-end correctness for the actual bucket driver on a structural strong plan.
The next module constructs such plans from arbitrary finite no-forgetting decision lists. -/
theorem solvePlan_spec (κ : id.Kernel R) (hclosed : id.Closed) (ord : RankedOrder id)
(hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
(hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
(plan : Plan id Finset.univ) :
let sol := solvePlan κ hloc hnonneg u hu plan
sol.strategy.Deterministic ∧ expectedUtility κ sol.strategy u = sol.value ∧
sol.value = optimalValue κ u := by
let result := run plan (initial κ hloc hnonneg u hu) (defaultStrategy (id := id))
have hc : Correct κ u result.1 result.2 := run_correct hclosed ord hloc hnorm plan _ _
(initial_correct κ hloc hnonneg u hu _)
have hd : result.2.Deterministic :=
run_deterministic plan _ defaultStrategy_deterministic
have hp : (collect result.1.valuations).prob (baseAssignment id) = 1 := by
rw [hc.mass, Finset.compl_empty, freeJoint_univ, marg_univ]
exact sum_joint_instantiate_eq_one κ result.2 hclosed ord.order hloc hnorm
have hr : expectedUtility κ result.2 u =
(collect result.1.valuations).util (baseAssignment id) := by
have h := hc.realizes (baseAssignment id)
rw [weight, hp, one_mul, Finset.compl_empty, freeJoint_univ, marg_univ] at h
exact h.symm
have ho : ∀ σ : Strategy id R, σ.Nonneg →
expectedUtility κ σ u ≤ (collect result.1.valuations).util (baseAssignment id) := by
intro σ hσ
have h := hc.dominates σ hσ (baseAssignment id)
rw [weight, hp, one_mul, Finset.compl_empty, freeJoint_univ, marg_univ] at h
exact h

```

```

change result.2.Deterministic  $\wedge$  expectedUtility  $\kappa$  result.2 u =
  (collect result.1.valuations).util (baseAssignment id)  $\wedge$ 
  (collect result.1.valuations).util (baseAssignment id) = optimalValue  $\kappa$  u
refine {hd, hr, le_antisymm ?_ ?_}
• rw [← hr]
  exact expectedUtility_le_optimalValue  $\kappa$  u result.2 (deterministic_nonneg _ hd)
• obtain { $\sigma$ , _, h $\sigma$ , hvalue} := optimalValue_attained  $\kappa$  u
  rw [← hvalue]
  exact ho  $\sigma$  h $\sigma$ 

end
end InfluenceDiagramsProofs.DVE

```

No-forgetting generates the strong schedule

```
import InfluenceDiagramsProofs.Finite.DVE.Solver
```

The input is an ordinary duplicate-free complete decision list, in reverse chronological order, satisfying perfect recall. At each decision, all variables outside its information set and action are summed out, then that action is maximised. The remaining variables are exactly its information set. No-forgetting proves that the summed block contains only chance variables and that recursion can continue. The final block contains no actions.

This is the source's first-observation block construction in reverse. The finite enumeration within a block is arbitrary; numerical correctness does not depend on min-fill.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram

noncomputable section

variable {id : FinInfluenceDiagram}

structure NoForgettingOrder (id : FinInfluenceDiagram) where
  reverseDecisions : List id.D
  nodup : reverseDecisions.Nodup
  complete :  $\forall$  d, d  $\in$  reverseDecisions
  remembers : reverseDecisions.Pairwise
    (fun later earlier  $\Rightarrow$  insert (id.action earlier) (id.info earlier)  $\subseteq$  id.info later)

def prependChances (xs : List id.V) (hn : xs.Nodup) {R : Finset id.V}
  (hR : xs.toFinset  $\subseteq$  R) (hc :  $\forall$  v  $\in$  xs,  $\forall$  d, id.action d  $\neq$  v)
  (next : Plan id (R \ xs.toFinset)) : Plan id R := by
induction xs generalizing R with
| nil  $\Rightarrow$  simp using next
| cons v xs ih  $\Rightarrow$ 
  have hv : v  $\in$  R := hR (by simp)
  have htail : xs.toFinset  $\subseteq$  R.erase v := by
    intro w hw
    refine Finset.mem_erase.2 {?, hR (by simp [List.mem_toFinset.1 hw])}
    intro he
    exact (List.nodup_cons.1 hn).1 (he  $\triangleright$  List.mem_toFinset.1 hw)
  have heq : R.erase v \ xs.toFinset = R \ (v :: xs).toFinset := by
    ext w
    by_cases hw : w = v <|> simp [hw]
  apply Plan.chance v hv (hc v (List.mem_cons_self ..))
  exact ih (List.nodup_cons.1 hn).2 htail
    (fun w hw  $\Rightarrow$  hc w (List.mem_cons_of_mem v hw)) (by simp [heq] using next)

/-- Structural construction, with no numerical assumptions or semantic invariant fields. -/
def buildPlan (ds : List id.D)
  (hnf : ds.Pairwise (fun d e  $\Rightarrow$  insert (id.action e) (id.info e)  $\subseteq$  id.info d))

```

```

(hself : ∀ d, id.action d ≠ id.info d) (R : Finset id.V)
(ha : ∀ d, id.action d ∈ R ↔ d ∈ ds) (hi : ∀ d ∈ ds, id.info d ⊆ R) :
Plan id R := by
induction ds generalizing R with
| nil =>
  apply prependChances R.toList (Finset.nodup_toList R) (by simp)
  · intro v hv d he
    have h := (ha d).1 (he ▸ Finset.mem_toList.1 hv)
    simp at h
  · simp using (Plan.done (id := id))
| cons d ds ih =>
  have hp := List.pairwise_cons.1 hnf
  have hd : id.action d ∈ R := (ha d).2 (List.mem_cons_self ..)
  have hI : id.info d ⊆ R := hi d (List.mem_cons_self ..)
  have hat : ∀ e, id.action e ∈ id.info d ↔ e ∈ ds := by
    intro e
    constructor
    · intro he
      rcases List.mem_cons.1 ((ha e).1 (hI he)) with rfl | ht
      · exact False.elim (hself e he)
      · exact ht
    · intro he
      exact hp.1 e he (Finset.mem_insert_self _ _)
  have hit : ∀ e ∈ ds, id.info e ⊆ id.info d := by
    intro e he v hv
    exact hp.1 e he (Finset.mem_insert_of_mem hv)
  have tail := ih hp.2 (id.info d) hat hit
  let K := insert (id.action d) (id.info d)
  let B := R \ K
  have hK : K ⊆ R := Finset.insert_subset_iff.2 ⟨hd, hI⟩
  have hrem : R \ B = K := by
    ext v
    by_cases hr : v ∈ R <|> by_cases hk : v ∈ K <|> simp_all [B]
  have he : K.erase (id.action d) = id.info d := Finset.erase_insert (hself d)
  have pd : Plan id K := Plan.decision d (Finset.mem_insert_self _ _) he
  (by simp [he] using tail)
  apply prependChances B.toList (Finset.nodup_toList B)
  (by
    intro v hv
    exact (Finset.mem_sdiff.1 (Finset.mem_toList.1 (List.mem_toFinset.1 hv))).1)
  · intro v hv e hev
    have hvB := Finset.mem_toList.1 hv
    have hvR := (Finset.mem_sdiff.1 hvB).1
    have hnot := (Finset.mem_sdiff.1 hvB).2
    have heR : id.action e ∈ R := hev.sym ▸ hvR
    rcases List.mem_cons.1 ((ha e).1 heR) with rfl | het
    · exact hnot (hev ▸ Finset.mem_insert_self _ _)
    · exact hnot (hev ▸ Finset.mem_insert_of_mem (hat e ▸.2 het))
  · simp [hrem] using pd

def NoForgettingOrder.plan (nf : NoForgettingOrder id) (hself : ∀ d, id.action d ≠ id.info d) :
Plan id Finset.univ :=
buildPlan nf.reverseDecisions nf.remembers hself Finset.univ
(fun d => by simp [nf.complete d]) (fun _ => Finset.subset_univ _)

theorem idxOf_lt_of_no_reverse {A : Type} [DecidableEq A] (l : List A) {a b : A}
(ha : a ∈ l) (hb : b ∈ l) (hne : a ≠ b)
(hp : l.Pairwise (fun x y => x = b → y ≠ a)) : l.idxOf a < l.idxOf b := by
induction l with
| nil => simp at ha
| cons c l ih =>
  by_cases hac : a = c
  · subst a
    simp [hne]
  · by_cases hbc : b = c
    · subst b

```

```

    have hal : a ∈ l := (List.mem_cons.1 ha).resolve_left hac
    exact False.elim ((List.pairwise_cons.1 hp).1 a hal rfl rfl)
  • have hal : a ∈ l := (List.mem_cons.1 ha).resolve_left hac
    have hbl : b ∈ l := (List.mem_cons.1 hb).resolve_left hbc
    simp [List.idxOf_cons, Ne.symm hac, Ne.symm hbc] using
      ih hal hbl (List.pairwise_cons.1 hp).2

/-- The rank witness is constructed from the existing model's topological order. -/
def RankedOrder.ofOrder (ord : id.IDOrder) : RankedOrder id where
  order := ord
  rank v := ord.order.idxOf v
  parents_lt m v hv := by
    apply idxOf_lt_of_no_reverse ord.order (ord.complete v) (ord.complete (id.target m))
    (fun he ⇒ ord.no_self m (he ▸ hv))
  exact ord.parents_before.imp (by
    intro x y h hx hy
    exact h m hx.symm (hy.symm ▸ hv))
  info_lt d v hv := by
    apply idxOf_lt_of_no_reverse ord.order (ord.complete v) (ord.complete (id.action d))
    (fun he ⇒ ord.no_self_info d (he ▸ hv))
  exact ord.info_before_action.imp (by
    intro x y h hx hy
    exact h d hx.symm (hy.symm ▸ hv))

/-- The solver obtains its schedule from perfect recall, not from a user-supplied legality
certificate. Choices are local utility argmaxes; no exhaustive strategy search is used. -/
def solve (κ : id.Kernel R) (hloc : ∀ m, Local κ m) (hnonneg : ∀ m x a, 0 ≤ κ m x a)
  (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) : Solution id :=
  solvePlan κ hloc hnonneg u hu (nf.plan ord.no_self_info)

/-- Finite multi-decision DVE correctness, no evidence. The generated strong-order bucket
driver returns an admissible deterministic strategy, realizes its reported value, and attains
the existing global optimum over all nonnegative strategies. Zero-probability rows are allowed.
This is exact arithmetic, not an assertion about Float64 tolerance or first-label tie identity. -/
theorem solve_spec (κ : id.Kernel R) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j) :
  let sol := solve κ hloc hnonneg u hu ord nf
  sol.strategy.Deterministic ∧ expectedUtility κ sol.strategy u = sol.value ∧
  sol.value = optimalValue κ u :=
  solvePlan_spec κ hclosed (RankedOrder.ofOrder ord) hloc hnorm hnonneg u hu _

end
end InfluenceDiagramsProofs.DVE

```

Action-independent evidence

```
import InfluenceDiagramsProofs.Finite.DVE.Schedule
```

Evidence is supported on a chance-ancestral set containing no action. This is a structural non-action-descendant condition, not an assumption that the likelihood normalizer is strategy-independent. That independence is proved below. Nonnegative likelihood weights include hard evidence indicators. Conditional claims require strictly positive evidence mass.

```
set_option autoImplicit false
```

```
namespace InfluenceDiagramsProofs.DVE
```

```
open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation
```

```
noncomputable section
```

```

variable {id : FinInfluenceDiagram}

structure Evidence (id : FinInfluenceDiagram) where
  ancestors : Finset id.V
  closed : ∀ m, id.target m ∈ ancestors → id.parents m ⊆ ancestors
  no_action : ∀ d, id.action d ∉ ancestors
  likelihood : id.Assignment → ℝ
  localOn : Depends ancestors likelihood
  nonneg : ∀ x, 0 ≤ likelihood x

namespace Evidence

/-- Hard observations on any subset of an action-free ancestral set. -/
def hard (A O : Finset id.V) (hO : O ⊆ A)
  (hc : ∀ m, id.target m ∈ A → id.parents m ⊆ A)
  (ha : ∀ d, id.action d ∉ A) (observed : id.Assignment) : Evidence id where
  ancestors := A
  closed := hc
  no_action := ha
  likelihood x := if ∀ v ∈ O, x v = observed v then 1 else 0
  localOn x y h := by
    dsimp only
    have he : (∀ v ∈ O, x v = observed v) ↔ (∀ v ∈ O, y v = observed v) := by
      constructor
      · intro hx v hv
        rw [← h v (hO hv)]
        exact hx v hv
      · intro hy v hv
        rw [h v (hO hv)]
        exact hy v hv
    simp only [he]
  nonneg x := by
    split_ifs
    · exact zero_le_one
    · exact le_refl 0

def valuation (e : Evidence id) : Valuation id.toFinBayesNet where
  scope := e.ancestors
  prob := e.likelihood
  util _ := 0
  prob_local := e.localOn
  util_local := fun _ _ _ => rfl
  nonneg := e.nonneg

/-- Put the action-free ancestral evidence block before every action. The original topological
order remains available; the new ranks preserve every causal and information inequality. -/
def ranked (e : Evidence id) (ord : id.IDOrder) : RankedOrder id where
  order := ord
  rank v := if v ∈ e.ancestors then ord.order.indexOf v else ord.order.length + ord.order.indexOf v
  parents_lt m v hv := by
    have hold := (RankedOrder.ofOrder ord).parents_lt m v hv
    have hlen := List.indexOf_lt_length_of_mem (ord.complete v)
    by_cases ht : id.target m ∈ e.ancestors
    · have hp := e.closed m ht hv
      simpa [ht, hp] using hold
    · by_cases hp : v ∈ e.ancestors
      · simp only [ht, hp, ↓reduceIte]
        exact hlen.trans_le (Nat.le_add_right _ _)
      · simpa [ht, hp] using Nat.add_lt_add_left hold ord.order.length
  info_lt d v hv := by
    have hold := (RankedOrder.ofOrder ord).info_lt d v hv
    have hlen := List.indexOf_lt_length_of_mem (ord.complete v)
    by_cases hp : v ∈ e.ancestors
    · simp only [e.no_action, hp, ↓reduceIte]
      exact hlen.trans_le (Nat.le_add_right _ _)
    · simpa [e.no_action, hp] using Nat.add_lt_add_left hold ord.order.length

```

```

theorem rank_before_actions (e : Evidence id) (ord : id.IDOrder) (d : id.D)
  (v : id.V) (hv : v ∈ e.ancestors) :
  (e.ranked ord).rank v < (e.ranked ord).rank (id.action d) := by
simp only [ranked, hv, e.no_action, ↓reduceIte]
exact (List.indexOf_lt_length_of_mem (ord.complete v)).trans_le (Nat.le_add_right _ _)

theorem probability_independent (e : Evidence id) (κ : id.Kernel R)
  (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (E : Finset id.V) (σ : Strategy id R) (d : id.D) (hd : id.action d ∉ E)
  (hboundary : Ec ⊆ insert (id.action d) (id.info d))
  (x : id.Assignment) (a : id.states (id.action d)) :
  marg E (fun y ⇒ freeJoint κ E σ y * e.likelihood y) (Function.update x (id.action d) a) =
  marg E (fun y ⇒ freeJoint κ E σ y * e.likelihood y) x := by
apply probability_independent_weighted κ hclosed (e.ranked ord) hloc hnorm E σ d hd hboundary
· intro y z h
  apply e.localOn
  intro v hv
  exact h v (Finset.mem_filter.2 ⟨Finset.mem_univ _, (e.rank_before_actions ord d v hv).le⟩)
· intro y b
  apply e.localOn
  intro v hv
  exact Function.update_of_ne (show v ≠ id.action d from fun he ⇒ e.no_action d (he ▶ hv)) _ _

end Evidence

def evidenceMass (κ : id.Kernel R) (σ : Strategy id R) (e : Evidence id) : R :=
  ∑ x, joint (strategyKernel κ σ) x * e.likelihood x

def evidenceNumerator (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R)
  (e : Evidence id) : R :=
  ∑ x, joint (strategyKernel κ σ) x * e.likelihood x * totalUtility u x

def conditionalEU (κ : id.Kernel R) (σ : Strategy id R) (u : Utility id R)
  (e : Evidence id) : R := evidenceNumerator κ σ u e / evidenceMass κ σ e

theorem evidenceMass_nonneg (e : Evidence id) (κ : id.Kernel R)
  (hκ : ∀ m x a, 0 ≤ κ m x a) (σ : Strategy id R) (hσ : σ.Nonneg) :
  0 ≤ evidenceMass κ σ e := by
apply Finset.sum_nonneg
intro x _
apply mul_nonneg _ (e.nonneg x)
rw [joint_instantiate]
exact mul_nonneg (Finset.prod_nonneg fun m _ ⇒ hκ m x _)
  (Finset.prod_nonneg fun d _ ⇒ hσ d x _)

/-- Evidence mass is independent of all policy choices, derived from action-free ancestry and
normalized downstream mechanisms/policies. Positivity is not assumed by this equality. -/
theorem evidenceMass_independent (e : Evidence id) (κ : id.Kernel R) (hclosed : id.Closed)
  (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (σ τ : Strategy id R) : evidenceMass κ σ e = evidenceMass κ τ e := by
let U := e.ancestors
have hU : UpstreamClosed (bn := id.instantiate) U := by
  rintro (m | d) hm
  · exact e.closed m hm
  · exact False.elim (e.no_action d hm)
have hd (ρ : Strategy id R) (x : id.Assignment) :
  marg (bn := id.instantiate) Uc (joint (strategyKernel κ ρ)) x =
  ∏ m ∈ Finset.univ.filter (fun m ⇒ id.instantiate.target m ∈ U),
  strategyKernel κ ρ m x (x (id.instantiate.target m)) :=
  marg_joint_upstream _ (closed_instantiate hclosed) ord.toTopoOrder
  (local_instantiate κ ρ hloc) (normalised_instantiate κ ρ hnorm) U hU x
have heq (x : id.Assignment) :
  marg (bn := id.instantiate) Uc (joint (strategyKernel κ σ)) x =
  marg (bn := id.instantiate) Uc (joint (strategyKernel κ τ)) x := by

```

```

rw [hd, hd]
apply Finset.prod_congr rfl
intro m hm
cases m with
| inl m => rfl
| inr d => exact False.elim (e.no_action d (Finset.mem_filter.1 hm).2)
have hw (ρ : Strategy id R) (x : id.Assignment) :
  marg (bn := id.instantiate) Uc (fun y => joint (strategyKernel κ ρ) y * e.likelihood y) x =
    e.likelihood x * marg (bn := id.instantiate) Uc (joint (strategyKernel κ ρ)) x := by
  have hf : (fun y => joint (bn := id.instantiate) (strategyKernel κ ρ) y * e.likelihood y) =
    (fun y => e.likelihood y * joint (bn := id.instantiate) (strategyKernel κ ρ) y) := by
    funext y
    ring
  rw [hf]
  exact marg_mul_left fun y hy => e.localOn y x fun v hv =>
    mem_fibre.1 hy v (fun hc => Finset.mem_compl.1 hc hv)
have hs (ρ : Strategy id R) :
  evidenceMass κ ρ e =
    marg U (fun x => e.likelihood x *
      marg (bn := id.instantiate) Uc (joint (strategyKernel κ ρ)) x)
    (baseAssignment id) := by
  calc
  - = marg (bn := id.instantiate) Finset.univ
    (fun y => joint (strategyKernel κ ρ) y * e.likelihood y) (baseAssignment id) :=
    (marg_univ _ _).symm
  - = marg (bn := id.instantiate) (U U Uc)
    (fun y => joint (strategyKernel κ ρ) y * e.likelihood y) (baseAssignment id) :=
    congrArg (fun S => marg (bn := id.instantiate) S
      (fun y => joint (strategyKernel κ ρ) y * e.likelihood y) (baseAssignment id))
      (Finset.union_compl U).symm
  - = marg U (fun x => marg (bn := id.instantiate) Uc
    (fun y => joint (strategyKernel κ ρ) y * e.likelihood y) x) (baseAssignment id) :=
    marg_union_disjoint (Finset.disjoint_left.2 (fun _ hu hc => Finset.mem_compl.1 hc hu)) _ _
  - = _ := congrArg (fun f => marg (bn := id.instantiate) U f (baseAssignment id)) (funext (hw
    ρ))
  rw [hs, hs]
  simp_rw [heq]

structure WeightedCorrect (κ : id.Kernel R) (u : Utility id R) (e : Evidence id)
  {R : Finset id.V} {s : State id R} {σ : Strategy id R} : Prop where
  mass : ∀ x, (collect s.valuations).prob x =
    marg Rc (fun y => freeJoint κ Rc σ y * e.likelihood y) x
  realizes : ∀ x, (collect s.valuations).weight x =
    marg Rc (fun y => freeJoint κ Rc σ y * (e.likelihood y * totalUtility u y)) x
  dominates : ∀ τ : Strategy id R, τ.Nonneg → ∀ x,
    marg Rc (fun y => freeJoint κ Rc τ y * (e.likelihood y * totalUtility u y)) x ≤
    (collect s.valuations).weight x

theorem WeightedCorrect.chance {κ : id.Kernel R} {u : Utility id R} {e : Evidence id}
  {R : Finset id.V} {s : State id R} {σ : Strategy id R} (h : WeightedCorrect κ u e s σ)
  (v : id.V) (hv : v ∈ R) (hc : ∀ d, id.action d ≠ v) :
  WeightedCorrect κ u e (s.chance v) σ := by
  have hn : v ∉ Rc := by simp using hv
  constructor
  • intro x
    change (collect (chanceStep v s.valuations)).prob x = _
    rw [chanceStep_prob, compl_erase, freeJoint_insert_chance κ Rc σ v hc, marg_insert hn]
    exact Finset.sum_congr rfl fun b _ => h.mass _
  • intro x
    change (collect (chanceStep v s.valuations)).weight x = _
    rw [chanceStep_weight, compl_erase, freeJoint_insert_chance κ Rc σ v hc, marg_insert hn]
    exact Finset.sum_congr rfl fun b _ => h.realizes _
  • intro τ ht x
    change _ ≤ (collect (chanceStep v s.valuations)).weight x
    rw [chanceStep_weight, compl_erase, freeJoint_insert_chance κ Rc τ v hc, marg_insert hn]
    exact Finset.sum_le_sum fun b _ => h.dominates τ ht _

```

```

theorem WeightedCorrect.decision {κ : id.Kernel ℝ} {u : Utility id ℝ} {e : Evidence id}
  {R : Finset id.V} {s : State id R} {σ : Strategy id R} (h : WeightedCorrect κ u e s σ)
  (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (d : id.D) (hd : id.action d ∈ R) (hi : R.erase (id.action d) = id.info d) :
  WeightedCorrect κ u e (s.decision d) (Function.update σ d (s.policy d hi)) := by
have hn : id.action d ∉ Rc := by simp using hd
have hincj := (id.closed_iff.1 hclosed).2.1
have hinfo := info_disjoint d hi
let f := choice (id.action d) (collect (bucket (id.action d) s.valuations))
have hpi : ∀ x a, (s.policy d hi).kernel x a = if a = f x then 1 else 0 := by
  intro x a
  rfl
have heval : ∀ (F : id.Assignment → ℝ) (x : id.Assignment),
  marg (insert (id.action d) Rc)
  (fun y ⇒ freeJoint κ (insert (id.action d) Rc)
    (Function.update σ d (s.policy d hi)) y * F y) x =
  marg Rc (fun y ⇒ freeJoint κ Rc σ y * F y) (Function.update x (id.action d) (f x)) := by
  intro F x
  rw [decision_marginal κ Rc σ hincj d hn hinfo]
  simp [hpi, ite_mul]
have hp : ∀ x a, (collect s.valuations).prob (Function.update x (id.action d) a) =
  (collect s.valuations).prob x := by
  intro x a
  rw [h.mass, h.mass]
exact e.probability_independent κ hclosed ord hloc hnorm Rc σ d hn
  (by simp using information_boundary d hi) x a
constructor
• intro x
  change (collect (decisionStep (id.action d) s.valuations)).prob x = _
  rw [(decisionStep_eval _ _ hp x).1, h.mass, compl_erase, heval]
• intro x
  change (collect (decisionStep (id.action d) s.valuations)).weight x = _
  rw [weight, (decisionStep_eval _ _ hp x).1, (decisionStep_eval _ _ hp x).2]
  change (collect s.valuations).weight (Function.update x (id.action d) (f x)) = _
  rw [h.realizes, compl_erase, heval]
• intro τ ht x
  change _ ≤ (collect (decisionStep (id.action d) s.valuations)).weight x
  rw [compl_erase]
have hc := decision_marginal κ Rc τ hincj d hn hinfo (τ d)
  (fun y ⇒ e.likelihood y * totalUtility u y) x
simp only [Function.update_eq_self] at hc
rw [hc]
calc
  _ ≤ ∑ a, (τ d).kernel x a * (collect (decisionStep (id.action d) s.valuations)).weight x :=
  Finset.sum_le_sum fun a _ => mul_le_mul_of_nonneg_left
    ((h.dominates τ ht _).trans (decisionStep_dominates _ _ hp x a)) (ht d x a)
  _ = _ := by rw [← Finset.sum_mul, (τ d).normalised x, one_mul]

theorem run_weighted_correct {κ : id.Kernel ℝ} {u : Utility id ℝ} {e : Evidence id}
  (hclosed : id.Closed) (ord : id.IDOrder) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R)
  (h : WeightedCorrect κ u e s σ) :
  WeightedCorrect κ u e (run plan s σ).1 (run plan s σ).2 := by
induction plan generalizing σ with
| done => exact h
| chance v hv hc next ih => exact ih (s.chance v) σ (h.chance v hv hc)
| decision d hd hi next ih =>
  exact ih (s.decision d) _ (h.decision hclosed ord hloc hnorm d hd hi)

def initialEvidence (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) : State id Finset.univ where
valuations := e.valuation :: (initial κ hloc hnonneg u hu).valuations
supported := Finset.subset_univ _

```

```

theorem initialEvidence_correct (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) (σ : Strategy id R) :
  WeightedCorrect κ u e (initialEvidence κ hloc hnonneg u hu e) σ := by
constructor
• intro x
  simp [initialEvidence, collect, combine, Evidence.valuation, initial_prob,
    marg_empty, freeJoint_empty, mul_comm]
• intro x
  simp [initialEvidence, collect, combine, Evidence.valuation, initial_prob, initial_util,
    marg_empty, freeJoint_empty, weight]
  ring
• intro τ _ x
  simp [initialEvidence, collect, combine, Evidence.valuation, initial_prob, initial_util,
    marg_empty, freeJoint_empty, weight]
  exact le_of_eq (by ring)

def solveEvidence (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (e : Evidence id) : Solution id :=
let result := run (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e)
  ↪ defaultStrategy
  ⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩

/-- **DVE with action-independent evidence.** Positive evidence mass is the explicit boundary.
The output policy realizes its conditional value and dominates every admissible stochastic
competitor. Zero information rows inside a positive-mass evidence event remain allowed. -/
theorem solveEvidence_spec (κ : id.Kernel R) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) (hpositive : 0 < evidenceMass κ defaultStrategy e) :
  let sol := solveEvidence κ hloc hnonneg u hu ord nf e
  sol.strategy.Deterministic ∧ conditionalEU κ sol.strategy u e = sol.value ∧
  ∀ σ : Strategy id R, σ.Nonneg → conditionalEU κ σ u e ≤ sol.value := by
let result := run (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e)
  ↪ defaultStrategy
have hc : WeightedCorrect κ u e result.1 result.2 := run_weighted_correct hclosed ord hloc hnorm
  _ _ _ (initialEvidence_correct κ hloc hnonneg u hu e _)
have hd : result.2.Deterministic :=
  run_deterministic _ _ _ defaultStrategy_deterministic
let Z := evidenceMass κ defaultStrategy e
have hp : (collect result.1.valuations).prob (baseAssignment id) = Z := by
  rw [hc.mass, Finset.compl_empty, freeJoint_univ, marg_univ]
  exact evidenceMass_independent e κ hclosed ord hloc hnorm result.2 defaultStrategy
have hr : evidenceNumerator κ result.2 u e =
  Z * (collect result.1.valuations).util (baseAssignment id) := by
  have h := hc.realizes (baseAssignment id)
  rw [weight, hp, Finset.compl_empty, freeJoint_univ, marg_univ] at h
  simp only [evidenceNumerator, mul_assoc] using h.symm
have ho : ∀ σ : Strategy id R, σ.Nonneg →
  evidenceNumerator κ σ u e ≤ Z * (collect result.1.valuations).util (baseAssignment id) := by
  intro σ hσ
  have h := hc.dominates σ hσ (baseAssignment id)
  rw [weight, hp, Finset.compl_empty, freeJoint_univ, marg_univ] at h
  simp only [evidenceNumerator, mul_assoc] using h
change result.2.Deterministic ∧ conditionalEU κ result.2 u e =
  (collect result.1.valuations).util (baseAssignment id) ∧
  ∀ σ : Strategy id R, σ.Nonneg → conditionalEU κ σ u e ≤
    (collect result.1.valuations).util (baseAssignment id)
refine ⟨hd, ?_, ?_⟩
• unfold conditionalEU
  rw [hr, evidenceMass_independent e κ hclosed ord hloc hnorm result.2 defaultStrategy]
  exact mul_div_cancel_left _ (ne_of_gt hpositive)
• intro σ hσ
  unfold conditionalEU

```

```

    rw [evidenceMass_independent e κ hclosed ord hloc hnorm σ defaultStrategy]
    apply (div_le_iff₀ hpositive).2
    simpa only [mul_comm] using ho σ hσ

/-- The evidence check uses the driver-computed mass, not an exhaustive oracle. -/
def solveEvidenceChecked (κ : id.Kernel ℝ) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m × a, 0 ≤ κ m × a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (e : Evidence id) : Option (Solution id) :=
  let result := run (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e)
  ↪ defaultStrategy
  if 0 < (collect result.1.valuations).prob (baseAssignment id) then
    some (collect result.1.valuations).util (baseAssignment id), result.2
  else none

theorem runEvidence_mass (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m × a, 0 ≤ κ m × a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) :
  let result := run (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e)
  ↪ defaultStrategy
  (collect result.1.valuations).prob (baseAssignment id) = evidenceMass κ defaultStrategy e := by
  dsimp only
  have hc := run_weighted_correct hclosed ord hloc hnorm (nf.plan ord.no_self_info)
    (initialEvidence κ hloc hnonneg u hu e) defaultStrategy
    (initialEvidence_correct κ hloc hnonneg u hu e _)
  rw [hc.mass, Finset.compl_empty, freeJoint_univ, marg_univ]
  exact evidenceMass_independent e κ hclosed ord hloc hnorm _ defaultStrategy

/-- Exact positive-mass boundary: the checked driver returns a solution precisely when
the evidence has positive mass. No choice of strategy can rescue a zero-mass event. -/
theorem solveEvidenceChecked_eq (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m × a, 0 ≤ κ m × a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) :
  solveEvidenceChecked κ hloc hnonneg u hu ord nf e =
    if 0 < evidenceMass κ defaultStrategy e then
      some (solveEvidence κ hloc hnonneg u hu ord nf e) else none := by
  unfold solveEvidenceChecked
  dsimp only
  rw [runEvidence_mass κ hclosed ord nf hloc hnorm hnonneg u hu e]
  rfl

/-- With valid nonnegative inputs, failure of the evidence check means exactly zero mass. -/
theorem solveEvidenceChecked_none_iff (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m × a, 0 ≤ κ m × a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) :
  solveEvidenceChecked κ hloc hnonneg u hu ord nf e = none ↔
    evidenceMass κ defaultStrategy e = 0 := by
  rw [solveEvidenceChecked_eq κ hclosed ord nf hloc hnorm hnonneg u hu e]
  by_cases hp : 0 < evidenceMass κ defaultStrategy e
  · simp [hp, ne_of_gt hp]
  · have hz : evidenceMass κ defaultStrategy e = 0 := le_antisymm (le_of_not_gt hp)
    (evidenceMass_nonneg e κ hnonneg _ (deterministic_nonneg _ defaultStrategy_deterministic))
    simp [hz]

def weightedUtility (u : Utility id ℝ) (e : Evidence id) : Utility id ℝ :=
  fun j x ⇒ e.likelihood x * u j x

theorem expectedUtility_weighted (κ : id.Kernel ℝ) (σ : Strategy id ℝ)
  (u : Utility id ℝ) (e : Evidence id) :
  expectedUtility κ σ (weightedUtility u e) = evidenceNumerator κ σ u e := by
  simp only [expectedUtility, weightedUtility, totalUtility, evidenceNumerator,
    ← Finset.mul_sum, mul_assoc]

/-- An independent global oracle, obtained by weighting utilities and dividing by the

```

```

strategy-independent evidence mass. This definition is not used by either driver. -/
def conditionalOptimalValue (κ : id.Kernel ℝ) (u : Utility id ℝ) (e : Evidence id) : ℝ :=
  optimalValue κ (weightedUtility u e) / evidenceMass κ defaultStrategy e

/-- The evidence driver equals the global oracle, under the same explicit positive-mass
and structural evidence assumptions as its realization/dominance theorem. -/
theorem solveEvidence_eq_optimal (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m × a, 0 ≤ κ m × a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) (hpositive : 0 < evidenceMass κ defaultStrategy e) :
  (solveEvidence κ hloc hnonneg u hu ord nf e).value = conditionalOptimalValue κ u e := by
let sol := solveEvidence κ hloc hnonneg u hu ord nf e
have hs := solveEvidence_spec κ hclosed ord nf hloc hnorm hnonneg u hu e hpositive
have he (σ : Strategy id ℝ) : conditionalEU κ σ u e =
  expectedUtility κ σ (weightedUtility u e) / evidenceMass κ defaultStrategy e := by
  rw [conditionalEU, expectedUtility_weighted,
    evidenceMass_independent e κ hclosed ord hloc hnorm σ defaultStrategy]
apply le_antisymm
· rw [← hs.2.1, he]
  exact div_le_div_of_nonneg_right
    (expectedUtility_le_optimalValue κ _ sol.strategy (deterministic_nonneg _ hs.1)) hpositive.le
· obtain {σ, _, hσ, hv} := optimalValue_attained κ (weightedUtility u e)
  rw [conditionalOptimalValue, ← hv, ← he]
  exact hs.2.2 σ hσ

end
end InfluenceDiagramsProofs.DVE

```

Nonvacuity: a partially observed two-decision model

```

import InfluenceDiagramsProofs.Finite.DVE.Evidence
import Mathlib.Tactic.FinCases

```

There are five Boolean variables: hidden state H, noisy observation O, first action T, second observation S, and final action A. H is never observed directly. If T is true, S reveals H; otherwise S is always false, creating unreachable information rows. The final action earns ten for matching H; testing costs one. The first decision sees O, while the second remembers O and T and also sees S. This is not a fully observed decision tree.

The generic driver theorem is instantiated with concrete normalized nonnegative CPTs, local utility, a closed diagram and a complete no-forgetting schedule.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE.Example

noncomputable section

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram

@[reducible] def diagram : FinInfluenceDiagram where
  V := Fin 5
  M := Fin 3
  states _ := Bool
  target m := if m = 0 then 0 else if m = 1 then 1 else 3
  parents m := if m = 0 then ∅ else if m = 1 then {0} else {0, 2}
  D := Bool
  action d := if d then 4 else 2
  info d := if d then {1, 2, 3} else {1}
  U := Unit
  uscope _ := {0, 2, 4}

theorem closed : diagram.Closed := by
  unfold FinInfluenceDiagram.Closed Function.Bijective Function.Injective Function.Surjective
  decide

```

```

def order : diagram.IDOrder where
  order := [0, 1, 2, 3, 4]
  nodup := by decide
  complete := by decide
  parents_before := by decide
  info_before_action := by decide
  no_self := by decide
  no_self_info := by decide

def noForgetting : NoForgettingOrder diagram where
  reverseDecisions := [true, false]
  nodup := by decide
  complete := by decide
  remembers := by decide

def kernels : diagram.Kernel R := fun (m : Fin 3) (x : Fin 5 → Bool) (a : Bool) =>
  if m = 0 then 1 / 2 else
  if m = 1 then (if a = x 0 then 3 / 4 else 1 / 4) else
  if x 2 then (if a = x 0 then 1 else 0) else (if a = false then 1 else 0)

theorem local_kernels : ∀ m, Local kernels m := by
  intro m
  fin_cases m
  · intro x y _
    rfl
  · intro x y h
    funext a
    have h0 := h 0 (by decide)
    simp [kernels, h0]
  · intro x y h
    funext a
    have h0 := h 0 (by decide)
    have h2 := h 2 (by decide)
    simp [kernels, h0, h2]

theorem normalised_kernels : ∀ m, Normalised kernels m := by
  intro m x
  change (∑ a : Bool, kernels m x a) = 1
  fin_cases m <=> cases hx : x 0 <=> cases ht : x 2 <=>
    norm_num [kernels, diagram, hx, ht, Fintype.sum_bool]

theorem nonnegative_kernels : ∀ m x a, 0 ≤ kernels m x a := by
  intro m x a
  unfold kernels
  split_ifs <=> norm_num

def utility : Utility diagram R := fun _ x =>
  (if x 4 = x 0 then 10 else 0) - (if x 2 then 1 else 0)

theorem local_utility : ∀ j, Utility.Local utility j := by
  intro j x y h
  have h0 := h 0 (by simp [diagram])
  have h2 := h 2 (by simp [diagram])
  have h4 := h 4 (by simp [diagram])
  simp [utility, h0, h2, h4]

theorem hidden_state_never_observed : ∀ d, (0 : diagram.V) ∉ diagram.info d := by decide

/-- Skipping the test makes S=true impossible, including information rows of decision A. -/
theorem unreachable_row (x : diagram.Assignment) (h : x 2 = false) :
  kernels 2 x true = 0 := by simp [kernels, h]

/-- Concrete multi-decision, partially observed, zero-row instantiation of DVE correctness. -/
theorem checked_solution :
  let sol := solve kernels local_kernels nonnegative_kernels utility local_utility order
  <- noForgetting

```

```

    sol.strategy.Deterministic ^ expectedUtility kernels sol.strategy utility = sol.value ^
    sol.value = optimalValue kernels utility :=
    solve_spec kernels closed order noForgetting local_kernels normalised_kernels
    nonnegative_kernels utility local_utility

end
end InfluenceDiagramsProofs.DVE.Example

```

The exact all-row guard and its positive-input intermediate lemma

```
import InfluenceDiagramsProofs.Finite.DVE.Evidence
```

ExactGuard is equality on every assignment and every action, not merely reachable rows. The checked driver uses the source-style maximum of row maximum-minus-minimum, proved equivalent to ExactGuard at exact zero tolerance. Strict positivity is used only in an intermediate cancellation argument; the final completeness theorem will remove it by continuous probability-expression provenance and normalized positive smoothing.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section

variable {id : FinInfluenceDiagram}

def ExactGuard (a : id.V) (vs : List (Valuation id.toFinBayesNet)) : Prop :=
  ∀ x b, (collect (bucket a vs)).prob (Function.update x a b) = (collect (bucket a vs)).prob x

def spread {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) : ℝ :=
  f (argmax f) - f (argmax fun a ⇒ -f a)

theorem spread_nonneg {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) : 0 ≤ spread f :=
  sub_nonneg.mpr (le_argmax f _)

theorem spread_zero_iff {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) :
  spread f = 0 ↔ ∀ a b, f a = f b := by
  constructor
  · intro h a b
    have he : f (argmax f) = f (argmax fun a ⇒ -f a) := sub_eq_zero.mp h
    have hlo (c : A) : f (argmax fun a ⇒ -f a) ≤ f c :=
      neg_le_neg_iff.mp (le_argmax (fun a ⇒ -f a) c)
    exact le_antisymm ((le_argmax f a).trans (he ▸ hlo b))
      ((le_argmax f b).trans (he ▸ hlo a))
  · intro h
    exact sub_eq_zero.mpr (h _ _)

def rowSpread (a : id.V) (vs : List (Valuation id.toFinBayesNet)) (x : id.Assignment) : ℝ :=
  spread fun b ⇒ (collect (bucket a vs)).prob (Function.update x a b)

def diagnosticSpread (a : id.V) (vs : List (Valuation id.toFinBayesNet)) : ℝ :=
  rowSpread a vs (argmax (rowSpread a vs))

/-- This is exactly the source diagnostic at atol=0: reject iff some row has positive
maximum-minus-minimum. No reachability restriction is hidden in the equivalence. -/
theorem exactGuard_iff_diagnostic (a : id.V) (vs : List (Valuation id.toFinBayesNet)) :
  ExactGuard a vs ↔ diagnosticSpread a vs ≤ 0 := by
  have hrow : ExactGuard a vs ↔ ∀ x, rowSpread a vs x = 0 := by
    constructor
    · intro h x
      apply (spread_zero_iff _).2
      intro b c

```

```

    exact (h x b).trans (h x c).symm
  • intro h x b
    have he := (spread_zero_iff _).1 (h x) b (x a)
    simpa only [Function.update_eq_self] using he
rw [hrow]
constructor
• intro h
  exact le_of_eq (h _)
• intro h x
  exact le_antisymm ((le_argmax (rowSpread a vs) x).trans h) (spread_nonneg _)

def AllGuards {R : Finset id.V} (plan : Plan id R)
  (vs : List (Valuation id.toFinBayesNet)) : Prop :=
  match plan with
  | .done => True
  | .chance v _ _ next => AllGuards next (chanceStep v vs)
  | .decision d _ _ next =>
    ExactGuard (id.action d) vs ∧ AllGuards next (decisionStep (id.action d) vs)

def checkedRun {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R) :
  Option (State id R × Strategy id R) := by
  classical
  exact match plan with
  | .done => some (s, σ)
  | .chance v _ _ next => checkedRun next (s.chance v) σ
  | .decision d _ hi next =>
    if 0 < diagnosticSpread (id.action d) s.valuations then none
    else checkedRun next (s.decision d) (Function.update σ d (s.policy d hi))

theorem checkedRun_eq_run {R : Finset id.V} (plan : Plan id R) (s : State id R)
  (σ : Strategy id R) (h : AllGuards plan s.valuations) :
  checkedRun plan s σ = some (run plan s σ) := by
  classical
  induction plan generalizing σ with
  | done => rfl
  | chance v _ _ next ih => exact ih (s.chance v) σ h
  | decision d _ hi next ih =>
    change (if 0 < diagnosticSpread (id.action d) s.valuations then none else _) = _
    rw [if_neg (not_lt.mpr ((exactGuard_iff_diagnostic _ _).1 h.1))]
    exact ih (s.decision d) _ h.2

def Positive {bn : FinBayesNet} (vs : List (Valuation bn)) : Prop :=
  ∀ v ∈ vs, ∀ x, 0 < v.prob x

theorem Positive.filter {bn : FinBayesNet} {vs : List (Valuation bn)} (h : Positive vs)
  (p : Valuation bn → Bool) : Positive (vs.filter p) :=
  fun v hv => h v (List.mem_of_mem_filter hv)

theorem Positive.collect {bn : FinBayesNet} {vs : List (Valuation bn)}
  (h : Positive vs) (x : bn.Assignment) : 0 < (Valuation.collect vs).prob x := by
  induction vs with
  | nil => exact zero_lt_one
  | cons v vs ih =>
    exact mul_pos (h v (List.mem_cons_self ..) x)
    (ih (fun w hw => h w (List.mem_cons_of_mem v hw)))

theorem Positive.chanceStep {bn : FinBayesNet} {vs : List (Valuation bn)}
  (h : Positive vs) (a : bn.V) : Positive (chanceStep a vs) := by
  intro v hv x
  rcases List.mem_cons.1 hv with rfl | hv
  • change 0 < ∑ b, (Valuation.collect (bucket a vs)).prob (Function.update x a b)
    exact Finset.sum_pos (fun b _ => (h.filter _).collect _) Finset.univ_nonempty
  • exact h v (List.mem_of_mem_filter hv) x

theorem Positive.decisionStep {bn : FinBayesNet} {vs : List (Valuation bn)}
  (h : Positive vs) (a : bn.V) : Positive (decisionStep a vs) := by

```

```

intro v hv x
rcases List.mem_cons.1 hv with rfl | hv
· exact (h.filter _).collect _
· exact h v (List.mem_of_mem_filter hv) x

theorem initial_positive (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (hp : ∀ m x a, 0 < κ m x a) :
  Positive (initial κ hloc hnonneg u hu).valuations := by
intro v hv x
rcases List.mem_append.1 hv with hv | hv
· obtain ⟨m, _, rfl⟩ := List.mem_map.1 hv
  exact hp m x _
· obtain ⟨j, _, rfl⟩ := List.mem_map.1 hv
  exact zero_lt_one

theorem initialEvidence_positive (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) (hp : ∀ m x a, 0 < κ m x a) (he : ∀ x, 0 < e.likelihood x) :
  Positive (initialEvidence κ hloc hnonneg u hu e).valuations := by
intro v hv x
rcases List.mem_cons.1 hv with rfl | hv
· exact he x
· exact initial_positive κ hloc hnonneg u hu hp v hv x

/-- Intermediate result only: positive probabilities make every outside product cancellable.
Its structural independence premise is derived from the existing causal/normalization theorem. -/
theorem guards_of_positive {κ : id.Kernel R} {u : Utility id R}
  (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R)
  (hc : Correct κ u s σ) (hp : Positive s.valuations) :
  AllGuards plan s.valuations := by
induction plan generalizing σ with
| done => trivial
| chance v hv hchance next ih =>
  exact ih (s.chance v) σ (hc.chance v hv hchance) (hp.chanceStep v)
| @decision R d hd hi next ih =>
  have hg : ∀ x a, (collect s.valuations).prob (Function.update x (id.action d) a) =
    (collect s.valuations).prob x := by
    intro x a
    rw [hc.mass, hc.mass]
    exact probability_independent κ hclosed (RankedOrder.ofOrder ord) hloc hnorm Rc σ d
    (by simpa using hd) (by simpa using information_boundary d hi) x a
  constructor
  · intro x a
    exact bucket_probability_constant_on_support _ _ hg x (ne_of_gt ((hp.filter _).collect x)) a
  · exact ih (s.decision d) _ (hc.decision hclosed (RankedOrder.ofOrder ord) hloc hnorm d hd hi)
    (hp.decisionStep _)

theorem guards_of_positive_weighted {κ : id.Kernel R} {u : Utility id R} {e : Evidence id}
  (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  {R : Finset id.V} (plan : Plan id R) (s : State id R) (σ : Strategy id R)
  (hc : WeightedCorrect κ u e s σ) (hp : Positive s.valuations) :
  AllGuards plan s.valuations := by
induction plan generalizing σ with
| done => trivial
| chance v hv hchance next ih =>
  exact ih (s.chance v) σ (hc.chance v hv hchance) (hp.chanceStep v)
| @decision R d hd hi next ih =>
  have hg : ∀ x a, (collect s.valuations).prob (Function.update x (id.action d) a) =
    (collect s.valuations).prob x := by
    intro x a
    rw [hc.mass, hc.mass]
    exact e.probability_independent κ hclosed ord hloc hnorm Rc σ d

```

```

      (by simp using hd) (by simp using information_boundary d hi) x a
    constructor
    • intro x a
      exact bucket_probability_constant_on_support _ _ hg x (ne_of_gt ((hp.filter _).collect x)) a
    • exact ih (s.decision d) _ (hc.decision hclosed ord hloc hnorm d hd hi) (hp.decisionStep _)
  end
end InfluenceDiagramsProofs.DVE

```

Probability-expression provenance

```

import InfluenceDiagramsProofs.Finite.DVE.Guard.Positive
import Mathlib.Topology.Algebra.Ring.Real
import Mathlib.Topology.Order.Lattice
import Mathlib.Topology.Order.DenselyOrdered
import Mathlib.Data.List.Forall2

```

Probability potentials use only original chance kernels, the optional likelihood, one, finite products, finite sums and finite maxima. Utility divisions and utility argmax choices do not enter this language. The symbolic bucket trace retains the actual sufficient scopes, so its partitions are exactly those of the numerical driver.

The maximum *value* is continuous even though a chosen maximizing action need not be. This distinction is essential for extending the positive-input guard identity to zero rows.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE.Guard

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section

variable {id : FinInfluenceDiagram}

inductive Expr (id : FinInfluenceDiagram)
| one
| chance (m : id.M)
| likelihood
| mul (f g : Expr id)
| sum (v : id.V) (f : Expr id)
| max (v : id.V) (f : Expr id)

def Expr.eval (κ : id.Kernel ℝ) (L : id.Assignment → ℝ) :
  Expr id → id.Assignment → ℝ
| .one, _ ⇒ 1
| .chance m, x ⇒ κ m x (x (id.target m))
| .likelihood, x ⇒ L x
| .mul f g, x ⇒ f.eval κ L x * g.eval κ L x
| .sum v f, x ⇒ ∑ a, f.eval κ L (Function.update x v a)
| .max v f, x ⇒ f.eval κ L
  (Function.update x v (argmax fun a ⇒ f.eval κ L (Function.update x v a)))

structure Symbolic (id : FinInfluenceDiagram) where
  scope : Finset id.V
  expr : Expr id

namespace Symbolic

def collect : List (Symbolic id) → Symbolic id
| [] ⇒ ⟨∅, .one⟩
| v :: vs ⇒
  ⟨v.scope ∪ (collect vs).scope, .mul v.expr (collect vs).expr⟩

def bucket (a : id.V) (vs : List (Symbolic id)) : List (Symbolic id) :=

```

```

vs.filter fun v => decide (a ∈ v.scope)

def outside (a : id.V) (vs : List (Symbolic id)) : List (Symbolic id) :=
  vs.filter fun v => decide (a ∉ v.scope)

def chanceStep (a : id.V) (vs : List (Symbolic id)) : List (Symbolic id) :=
  ⟨(collect (bucket a vs)).scope.erase a, .sum a (collect (bucket a vs)).expr⟩ :: outside a vs

def decisionStep (a : id.V) (vs : List (Symbolic id)) : List (Symbolic id) :=
  ⟨(collect (bucket a vs)).scope.erase a, .max a (collect (bucket a vs)).expr⟩ :: outside a vs

def trace {R : Finset id.V} (plan : Plan id R) (vs : List (Symbolic id)) :
  List (id.V × Expr id) :=
  match plan with
  | .done => []
  | .chance a _ _ next => trace next (chanceStep a vs)
  | .decision d _ _ next =>
    (id.action d, (collect (bucket (id.action d) vs)).expr) :: trace next (decisionStep (id.action d)
    vs)

def initial (id : FinInfluenceDiagram) : List (Symbolic id) :=
  Finset.univ.toList.map (fun m => ⟨insert (id.target m) (id.parents m), .chance m⟩) ++
  Finset.univ.toList.map (fun u => ⟨id.uscope u, .one⟩)

def initialEvidence (id : FinInfluenceDiagram) (A : Finset id.V) : List (Symbolic id) :=
  ⟨A, .likelihood⟩ :: initial id

end Symbolic

def Represents (κ : id.Kernel ℝ) (L : id.Assignment → ℝ)
  (v : Valuation id.toFinBayesNet) (s : Symbolic id) : Prop :=
  v.scope = s.scope ∧ ∀ x, v.prob x = s.expr.eval κ L x

def RepresentsList (κ : id.Kernel ℝ) (L : id.Assignment → ℝ)
  (vs : List (Valuation id.toFinBayesNet)) (ss : List (Symbolic id)) : Prop :=
  List.Forall₂ (Represents κ L) vs ss

theorem RepresentsList.collect {κ : id.Kernel ℝ} {L : id.Assignment → ℝ}
  {vs : List (Valuation id.toFinBayesNet)} {ss : List (Symbolic id)}
  (h : RepresentsList κ L vs ss) :
  Represents κ L (Valuation.collect vs) (Symbolic.collect ss) := by
  induction h with
  | nil => exact ⟨rfl, fun _ => rfl⟩
  | @cons v s vs ss hv hs ih =>
    constructor
    · exact congrArg₂ (fun S T : Finset id.V => S U T) hv.1 ih.1
    · intro x
      change v.prob x * (Valuation.collect vs).prob x =
        s.expr.eval κ L x * (Symbolic.collect ss).expr.eval κ L x
      rw [hv.2, ih.2]

theorem RepresentsList.filter {κ : id.Kernel ℝ} {L : id.Assignment → ℝ}
  {vs : List (Valuation id.toFinBayesNet)} {ss : List (Symbolic id)}
  (h : RepresentsList κ L vs ss) (p : Finset id.V → Bool) :
  RepresentsList κ L (vs.filter fun v => p v.scope) (ss.filter fun s => p s.scope) := by
  induction h with
  | nil => exact List.Forall₂.nil
  | @cons v s vs ss hv hs ih =>
    by_cases hp : p v.scope
    · simpa only [List.filter_cons, ← hv.1, hp, ↓reduceIte] using List.Forall₂.cons hv ih
    · simpa only [List.filter_cons, ← hv.1, hp, Bool.false_eq_true, ↓reduceIte] using ih

theorem RepresentsList.chanceStep {κ : id.Kernel ℝ} {L : id.Assignment → ℝ}
  {vs : List (Valuation id.toFinBayesNet)} {ss : List (Symbolic id)}
  (h : RepresentsList κ L vs ss) (a : id.V) :
  RepresentsList κ L (Valuation.chanceStep a vs) (Symbolic.chanceStep a ss) := by

```

```

have hb : Represents  $\kappa$  L (Valuation.collect (Valuation.bucket a vs))
  (Symbolic.collect (Symbolic.bucket a ss)) := (h.filter (fun S  $\Rightarrow$  decide (a  $\in$  S))).collect
apply List.Forall2.cons _ (h.filter (fun S  $\Rightarrow$  decide (a  $\notin$  S)))
constructor
• exact congrArg (Finset.erase · a) hb.1
• intro x
  exact Finset.sum_congr rfl fun b _  $\Rightarrow$  hb.2 (Function.update x a b)

theorem RepresentsList.decisionStep { $\kappa$  : id.Kernel  $\mathbb{R}$ } {L : id.Assignment  $\rightarrow$   $\mathbb{R}$ }
  {vs : List (Valuation id.toFinBayesNet)} {ss : List (Symbolic id)}
  (h : RepresentsList  $\kappa$  L vs ss) (a : id.V) :
  RepresentsList  $\kappa$  L (Valuation.decisionStep a vs) (Symbolic.decisionStep a ss) := by
have hb : Represents  $\kappa$  L (Valuation.collect (Valuation.bucket a vs))
  (Symbolic.collect (Symbolic.bucket a ss)) := (h.filter (fun S  $\Rightarrow$  decide (a  $\in$  S))).collect
apply List.Forall2.cons _ (h.filter (fun S  $\Rightarrow$  decide (a  $\notin$  S)))
constructor
• exact congrArg (Finset.erase · a) hb.1
• intro x
  change (Valuation.collect (Valuation.bucket a vs)).prob
    (Function.update x a (argmax fun b  $\Rightarrow$  (Valuation.collect (Valuation.bucket a vs)).prob
      (Function.update x a b))) = _
  simp_rw [hb.2]
  rfl

def Expr.guard ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (L : id.Assignment  $\rightarrow$   $\mathbb{R}$ ) (p : id.V  $\times$  Expr id) : Prop :=
   $\forall$  x a, p.2.eval  $\kappa$  L (Function.update x p.1 a) = p.2.eval  $\kappa$  L x

/-- The symbolic trace records every actual bucket guard, not a guessed or assumed trace. -/
theorem represents_guards_iff { $\kappa$  : id.Kernel  $\mathbb{R}$ } {L : id.Assignment  $\rightarrow$   $\mathbb{R}$ }
  {R : Finset id.V} (plan : Plan id R)
  {vs : List (Valuation id.toFinBayesNet)} {ss : List (Symbolic id)}
  (h : RepresentsList  $\kappa$  L vs ss) :
  AllGuards plan vs  $\Leftrightarrow \forall p \in$  Symbolic.trace plan ss, Expr.guard  $\kappa$  L p := by
induction plan generalizing vs ss with
| done  $\Rightarrow$  simp [AllGuards, Symbolic.trace]
| chance a _ _ next ih  $\Rightarrow$  exact ih (h.chanceStep a)
| decision d _ _ next ih  $\Rightarrow$ 
  have hb : Represents  $\kappa$  L (Valuation.collect (Valuation.bucket (id.action d) vs))
    (Symbolic.collect (Symbolic.bucket (id.action d) ss)) :=
    (h.filter (fun S  $\Rightarrow$  decide (id.action d  $\in$  S))).collect
  change (ExactGuard (id.action d) vs  $\wedge$  _)  $\Leftrightarrow$  _
  rw [show ExactGuard (id.action d) vs  $\Leftrightarrow$ 
    Expr.guard  $\kappa$  L (id.action d, (Symbolic.collect (Symbolic.bucket (id.action d) ss)).expr) by
    unfold ExactGuard Expr.guard
    simp_rw [hb.2]]
  rw [ih (h.decisionStep (id.action d))]
  simp only [Symbolic.trace, List.forall_mem_cons]

theorem initial_represents ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hloc :  $\forall m$ , Local  $\kappa$  m)
  (hnonneg :  $\forall m \times a$ ,  $0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j)
  (L : id.Assignment  $\rightarrow$   $\mathbb{R}$ ) :
  RepresentsList  $\kappa$  L (DVE.initial  $\kappa$  hloc hnonneg u hu).valuations (Symbolic.initial id) := by
apply List.rel_append
• rw [List.forall2_map_left_iff, List.forall2_map_right_iff, List.forall2_same]
  intro v hv
  exact (rfl, fun _  $\Rightarrow$  rfl)
• rw [List.forall2_map_left_iff, List.forall2_map_right_iff, List.forall2_same]
  intro v hv
  exact (rfl, fun _  $\Rightarrow$  rfl)

theorem initialEvidence_represents ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hloc :  $\forall m$ , Local  $\kappa$  m)
  (hnonneg :  $\forall m \times a$ ,  $0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j)
  (e : Evidence id) :
  RepresentsList  $\kappa$  e.likelihood (DVE.initialEvidence  $\kappa$  hloc hnonneg u hu e).valuations
    (Symbolic.initialEvidence id e.ancestors) :=
  List.Forall2.cons (rfl, fun _  $\Rightarrow$  rfl) (initial_represents  $\kappa$  hloc hnonneg u hu _)

```

```

theorem argmax_value_eq_sup {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) :
  f (argmax f) = Finset.univ.sup' Finset.univ_nonempty f := by
  apply le_antisymm
  · exact Finset.le_sup' f (Finset.mem_univ _)
  · exact Finset.sup'_le _ _ (fun a _ => le_argmax f a)

/-- Only maximum values, not argmax selectors, are claimed continuous. -/
theorem continuousAt_argmax_value {A : Type} [Fintype A] [Nonempty A]
  (f : ℝ → A → ℝ) {t : ℝ} (h : ∀ a, ContinuousAt (fun z => f z a) t) :
  ContinuousAt (fun z => f z (argmax (f z))) t := by
  simp_rw [argmax_value_eq_sup]
  exact ContinuousAt.finset_sup'_apply _ (fun a _ => h a)

theorem Expr.continuousAt (e : Expr id) (K : ℝ → id.Kernel ℝ)
  (L : ℝ → id.Assignment → ℝ) {t : ℝ}
  (hK : ∀ m x a, ContinuousAt (fun z => K z m x a) t)
  (hL : ∀ x, ContinuousAt (fun z => L z x) t) (x : id.Assignment) :
  ContinuousAt (fun z => e.eval (K z) (L z) x) t := by
  induction e generalizing x with
  | one => exact continuousAt_const
  | chance m => exact hK m x _
  | likelihood => exact hL x
  | mul f g ihf ihg => exact (ihf x).mul (ihg x)
  | sum v f ih => exact tendsto_finsetSum _ (fun a _ => ih (Function.update x v a))
  | max v f ih =>
    exact continuousAt_argmax_value (fun z a => f.eval (K z) (L z) (Function.update x v a))
      (fun a => ih _)

/-- Equality on all positive smoothing parameters extends to zero by continuity. -/
theorem eq_at_zero_of_positive (f g : ℝ → ℝ)
  (hf : ContinuousAt f 0) (hg : ContinuousAt g 0) (h : ∀ t, 0 < t → f t = g t) :
  f 0 = g 0 := by
  have hz : f 0 - g 0 = 0 :=
    ((hf.sub hg).continuousWithinAt (s := Set.Ioi 0)).eq_const_of_mem_closure
      (by simp [closure_Ioi]) (fun t ht => sub_eq_zero.mpr (h t ht))
  exact sub_eq_zero.mpr hz
end
end InfluenceDiagramsProofs.DVE.Guard

```

Exact all-row diagnostic completeness

```
import InfluenceDiagramsProofs.Finite.DVE.Guard.Provenance
```

For $t > 0$, Laplace smoothing makes every chance row strictly positive while preserving locality and normalization. Adding t to the action-free likelihood makes its factors positive too. The positive-case theorem therefore proves every exact guard along each smoothed run.

The probability-expression trace is fixed by the scopes and plan. Its finite products, sums and maximum values are continuous at $t=0$. Every guard equality consequently holds at $t=0$, including contexts where the outside probability is zero. No strict-positivity assumption survives in the public theorems, and no cancellation by zero is performed.

```
set_option autoImplicit false
```

```
namespace InfluenceDiagramsProofs.DVE
```

```
open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation
open Guard
```

```
noncomputable section
```

```

variable {id : FinInfluenceDiagram}

def smoothKernel (κ : id.Kernel ℝ) (t : ℝ) : id.Kernel ℝ :=
  fun m x a => (κ m x a + t) / (1 + (Fintype.card (id.states (id.target m)) : ℝ) * t)

theorem smoothing_denominator_pos (m : id.M) (t : ℝ) (ht : 0 ≤ t) :
  0 < 1 + (Fintype.card (id.states (id.target m)) : ℝ) * t :=
  add_pos_of_pos_of_nonneg zero_lt_one (mul_nonneg (Nat.cast_nonneg _) ht)

theorem smoothKernel_zero (κ : id.Kernel ℝ) : smoothKernel κ 0 = κ := by
  funext m x a
  simp [smoothKernel]

theorem smoothKernel_local (κ : id.Kernel ℝ) (hk : ∀ m, Local κ m) (t : ℝ) :
  ∀ m, Local (smoothKernel κ t) m := by
  intro m x y h
  funext a
  simp only [smoothKernel, hk m x y h]

theorem smoothKernel_normalised (κ : id.Kernel ℝ) (hk : ∀ m, Normalised κ m)
  (t : ℝ) (ht : 0 ≤ t) : ∀ m, Normalised (smoothKernel κ t) m := by
  intro m x
  change (∑ a, (κ m x a + t) / (1 + (Fintype.card (id.states (id.target m)) : ℝ) * t)) = 1
  simp only [div_eq_mul_inv]
  rw [← Finset.sum_mul, Finset.sum_add_distrib, hk m x]
  simp only [Finset.sum_const, Finset.card_univ, nsmul_eq_mul]
  exact mul_inv_cancel₀ (ne_of_gt (smoothing_denominator_pos m t ht))

theorem smoothKernel_positive (κ : id.Kernel ℝ) (hk : ∀ m x a, 0 ≤ κ m x a)
  (t : ℝ) (ht : 0 < t) : ∀ m x a, 0 < smoothKernel κ t m x a := by
  intro m x a
  exact div_pos (add_pos_of_nonneg_of_pos (hk m x a) ht) (smoothing_denominator_pos m t ht.le)

theorem smoothKernel_continuousAt_zero (κ : id.Kernel ℝ) (m : id.M)
  (x : id.Assignment) (a : id.states (id.target m)) :
  ContinuousAt (fun t => smoothKernel κ t m x a) 0 := by
  exact (continuousAt_const.add continuousAt_id).div
  (continuousAt_const.add (continuousAt_const.mul continuousAt_id)) (by simp)

def smoothEvidence (e : Evidence id) (t : ℝ) (ht : 0 ≤ t) : Evidence id where
  ancestors := e.ancestors
  closed := e.closed
  no_action := e.no_action
  likelihood x := e.likelihood x + t
  localOn x y h := by
    dsimp only
    rw [e.localOn x y h]
  nonneg x := add_nonneg (e.nonneg x) ht

/-- Every probability diagnostic in an actual no-evidence run holds on every row.
Only nonnegativity, not strict positivity, is assumed. The plan is structural. -/
theorem all_guards_complete (κ : id.Kernel ℝ) (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id ℝ) (hu : ∀ j, Utility.Local u j)
  (plan : Plan id Finset.univ) :
  AllGuards plan (initial κ hloc hnonneg u hu).valuations := by
  apply (represents_guards_iff plan (initial_represents κ hloc hnonneg u hu (fun _ => 1))).2
  intro p hp x a
  have ht : ∀ t : ℝ, 0 < t =>
    p.2.eval (smoothKernel κ t) (fun _ => 1) (Function.update x p.1 a) =
    p.2.eval (smoothKernel κ t) (fun _ => 1) x := by
    intro t ht
    let K := smoothKernel κ t
    have hl : ∀ m, Local K m := smoothKernel_local κ hloc t
    have hn : ∀ m, Normalised K m := smoothKernel_normalised κ hnorm t ht.le
    have hpos : ∀ m x a, 0 < K m x a := smoothKernel_positive κ hnonneg t ht

```

```

have hnn :  $\forall m \times a, 0 \leq K m \times a := \text{fun } m \times a \Rightarrow (\text{hpos } m \times a).\text{le}$ 
have hguards := guards_of_positive hclosed ord hl hn plan (initial K hl hnn u hu)
  defaultStrategy (initial_correct K hl hnn u hu _) (initial_positive K hl hnn u hu hpos)
exact (represents_guards_iff plan (initial_represents K hl hnn u hu (fun _  $\Rightarrow$  1))).1
  hguards p hp x a
have hx := p.2.continuousAt (smoothKernel  $\kappa$ ) (fun _  $\Rightarrow$  1)
  (smoothKernel_continuousAt_zero  $\kappa$ ) (fun _  $\Rightarrow$  continuousAt_const) x
have ha := p.2.continuousAt (smoothKernel  $\kappa$ ) (fun _  $\Rightarrow$  1)
  (smoothKernel_continuousAt_zero  $\kappa$ ) (fun _  $\Rightarrow$  continuousAt_const) (Function.update x p.1 a)
have hzero := eq_at_zero_of_positive _ _ ha hx ht
simp only [smoothKernel_zero] using hzero

/-- All-row completeness also holds for supported evidence, even if the entire evidence event
has zero mass. The later evidence-mass check is a different diagnostic. -/
theorem all_guards_complete_evidence ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hclosed : id.Closed) (ord : id.IDOrder)
  (hloc :  $\forall m$ , Local  $\kappa m$ ) (hnorm :  $\forall m$ , Normalised  $\kappa m$ )
  (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j)
  (e : Evidence id) (plan : Plan id Finset.univ) :
  AllGuards plan (initialEvidence  $\kappa$  hloc hnonneg u hu e).valuations := by
apply (represents_guards_iff plan (initialEvidence_represents  $\kappa$  hloc hnonneg u hu e)).2
intro p hp x a
have ht :  $\forall t : \mathbb{R}, 0 < t \rightarrow$ 
  p.2.eval (smoothKernel  $\kappa$  t) (fun y  $\Rightarrow$  e.likelihood y + t) (Function.update x p.1 a) =
  p.2.eval (smoothKernel  $\kappa$  t) (fun y  $\Rightarrow$  e.likelihood y + t) x := by
intro t ht
let K := smoothKernel  $\kappa$  t
let et := smoothEvidence e t ht.le
have hl :  $\forall m$ , Local K m := smoothKernel_local  $\kappa$  hloc t
have hn :  $\forall m$ , Normalised K m := smoothKernel_normalised  $\kappa$  hnrm t ht.le
have hpos :  $\forall m \times a, 0 < K m \times a := \text{smoothKernel\_positive } \kappa \text{ hnonneg } t \text{ ht}$ 
have hnn :  $\forall m \times a, 0 \leq K m \times a := \text{fun } m \times a \Rightarrow (\text{hpos } m \times a).\text{le}$ 
have hepos :  $\forall x, 0 < \text{et.likelihood } x := \text{fun } x \Rightarrow$ 
  add_pos_of_nonneg_of_pos (e.nonneg x) ht
have hguards := guards_of_positive_weighted hclosed ord hl hn plan
  (initialEvidence K hl hnn u hu et) defaultStrategy (initialEvidence_correct K hl hnn u hu et _)
  (initialEvidence_positive K hl hnn u hu et hpos hepos)
exact (represents_guards_iff plan (initialEvidence_represents K hl hnn u hu et)).1
  hguards p hp x a
have hl :  $\forall y$ , ContinuousAt (fun t :  $\mathbb{R} \Rightarrow$  e.likelihood y + t) 0 :=
  fun _  $\Rightarrow$  continuousAt_const.add continuousAt_id
have hx := p.2.continuousAt (smoothKernel  $\kappa$ ) (fun t y  $\Rightarrow$  e.likelihood y + t)
  (smoothKernel_continuousAt_zero  $\kappa$ ) hl x
have ha := p.2.continuousAt (smoothKernel  $\kappa$ ) (fun t y  $\Rightarrow$  e.likelihood y + t)
  (smoothKernel_continuousAt_zero  $\kappa$ ) hl (Function.update x p.1 a)
have hzero := eq_at_zero_of_positive _ _ ha hx ht
simp only [smoothKernel_zero, add_zero] using hzero

/-- The exact checked driver cannot reject a valid generated no-forgetting run. -/
theorem checkedRun_generated_eq ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc :  $\forall m$ , Local  $\kappa m$ ) (hnrm :  $\forall m$ , Normalised  $\kappa m$ )
  (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j) :
  checkedRun (nf.plan ord.no_self_info) (initial  $\kappa$  hloc hnonneg u hu) defaultStrategy =
  some (run (nf.plan ord.no_self_info) (initial  $\kappa$  hloc hnonneg u hu) defaultStrategy) :=
checkedRun_eq_run _ _ _ (all_guards_complete_evidence  $\kappa$  hclosed ord hloc hnrm hnonneg u hu _)

theorem checkedRun_generated_evidence_eq ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc :  $\forall m$ , Local  $\kappa m$ ) (hnrm :  $\forall m$ , Normalised  $\kappa m$ )
  (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j)
  (e : Evidence id) :
  checkedRun (nf.plan ord.no_self_info) (initialEvidence  $\kappa$  hloc hnonneg u hu e) defaultStrategy =
  some (run (nf.plan ord.no_self_info) (initialEvidence  $\kappa$  hloc hnonneg u hu e) defaultStrategy)
  :=
checkedRun_eq_run _ _ _ (all_guards_complete_evidence  $\kappa$  hclosed ord hloc hnrm hnonneg u hu e _)

def solveGuarded ( $\kappa$  : id.Kernel  $\mathbb{R}$ ) (hloc :  $\forall m$ , Local  $\kappa m$ )
  (hnonneg :  $\forall m \times a, 0 \leq \kappa m \times a$ ) (u : Utility id  $\mathbb{R}$ ) (hu :  $\forall j$ , Utility.Local u j)

```

```

(ord : id.IDOrder) (nf : NoForgettingOrder id) : Option (Solution id) :=
  (checkedRun (nf.plan ord.no_self_info) (initial κ hloc hnonneg u hu) defaultStrategy).map
    (fun result => ⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩)

theorem solveGuarded_eq (κ : id.Kernel R) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j) :
  solveGuarded κ hloc hnonneg u hu ord nf = some (solve κ hloc hnonneg u hu ord nf) := by
  unfold solveGuarded
  rw [checkedRun_generated_eq κ hclosed ord nf hloc hnorm hnonneg u hu]
  rfl

def solveEvidenceGuarded (κ : id.Kernel R) (hloc : ∀ m, Local κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (ord : id.IDOrder) (nf : NoForgettingOrder id) (e : Evidence id) : Option (Solution id) :=
  (checkedRun (nf.plan ord.no_self_info) (initialEvidence κ hloc hnonneg u hu e)
    ← defaultStrategy).bind
    (fun result => if 0 < (collect result.1.valuations).prob (baseAssignment id) then
      some ⟨(collect result.1.valuations).util (baseAssignment id), result.2⟩ else none)

/-- The guarded evidence solver has exactly the same result as the already-verified
evidence-mass-checked solver: no extra failure is introduced at unreachable rows. -/
theorem solveEvidenceGuarded_eq (κ : id.Kernel R) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) :
  solveEvidenceGuarded κ hloc hnonneg u hu ord nf e =
    solveEvidenceChecked κ hloc hnonneg u hu ord nf e := by
  unfold solveEvidenceGuarded
  rw [checkedRun_generated_evidence_eq κ hclosed ord nf hloc hnorm hnonneg u hu e]
  rfl

/-- The exact checked solver returns the same realized deterministic optimum; its diagnostic
introduces no additional hypothesis or failure on any valid no-evidence diagram. -/
theorem solveGuarded_spec (κ : id.Kernel R) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j) :
  ∃ sol : Solution id, solveGuarded κ hloc hnonneg u hu ord nf = some sol ∧
    sol.strategy.Deterministic ∧ expectedUtility κ sol.strategy u = sol.value ∧
    sol.value = optimalValue κ u :=
  ⟨solve κ hloc hnonneg u hu ord nf, solveGuarded_eq κ hclosed ord nf hloc hnorm hnonneg u hu,
    solve_spec κ hclosed ord nf hloc hnorm hnonneg u hu⟩

/-- For supported evidence, the only possible rejection is zero evidence mass, not the
exact probability diagnostic, even when many individual information rows are unreachable. -/
theorem solveEvidenceGuarded_none_iff (κ : id.Kernel R) (hclosed : id.Closed) (ord : id.IDOrder)
  (nf : NoForgettingOrder id) (hloc : ∀ m, Local κ m) (hnorm : ∀ m, Normalised κ m)
  (hnonneg : ∀ m x a, 0 ≤ κ m x a) (u : Utility id R) (hu : ∀ j, Utility.Local u j)
  (e : Evidence id) :
  solveEvidenceGuarded κ hloc hnonneg u hu ord nf e = none ↔
    evidenceMass κ defaultStrategy e = 0 := by
  rw [solveEvidenceGuarded_eq κ hclosed ord nf hloc hnorm hnonneg u hu e]
  exact solveEvidenceChecked_none_iff κ hclosed ord nf hloc hnorm hnonneg u hu e

end
end InfluenceDiagramsProofs.DVE

```

Exact-guard boundary checks

```

import InfluenceDiagramsProofs.Finite.DVE.Guard.Complete
import InfluenceDiagramsProofs.Finite.DVE.Example

```

The no-evidence fixture has a deterministic observed root Z, an action A that observes Z, and a deterministic child

Y of A. Summing Y retains an A-indexed probability factor. The root factor is outside A's bucket and is zero at $Z=\text{true}$. The complete diagram nevertheless has mass one and satisfies the exact guard on that unreachable row.

The previous hidden-state two-decision fixture is also checked with an identically zero supported likelihood. The probability guard still succeeds; only the later evidence-mass check rejects. These are nonvacuity checks, not substitutes for the general completeness proof.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs.DVE.GuardBoundary

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet FinInfluenceDiagram Valuation

noncomputable section

@[reducible] def diagram : FinInfluenceDiagram where
  V := Fin 3
  M := Bool
  states _ := Bool
  target m := if m then 2 else 0
  parents m := if m then {1} else ∅
  D := Unit
  action _ := 1
  info _ := {0}
  U := Unit
  uscope _ := {1}

theorem closed : diagram.Closed := by
  unfold FinInfluenceDiagram.Closed Function.Bijective Function.Injective Function.Surjective
  decide

def order : diagram.IDOrder where
  order := [0, 1, 2]
  nodup := by decide
  complete := by decide
  parents_before := by decide
  info_before_action := by decide
  no_self := by decide
  no_self_info := by decide

def nf : NoForgettingOrder diagram where
  reverseDecisions := [()]
  nodup := by decide
  complete := by decide
  remembers := by decide

def κ : diagram.Kernel ℝ := fun (m : Bool) (x : Fin 3 → Bool) (a : Bool) =>
  if m then (if a = x 1 then 1 else 0) else (if a = false then 1 else 0)

theorem κ_local : ∀ m, Local κ m := by
  intro m
  cases m with
  | false => intro x y _; rfl
  | true =>
    intro x y h
    funext a
    have he := h 1 (by decide)
    simp [κ, he]

theorem κ_normalised : ∀ m, Normalised κ m := by
  intro m x
  change (∑ a : Bool, κ m x a) = 1
  cases m <|> simp [κ]

theorem κ_nonnegative : ∀ m x a, 0 ≤ κ m x a := by
  intro m x a
  unfold κ

```

```

split_ifs <=> norm_num

def u : Utility diagram R := fun _ x => if x 1 then 1 else 0

theorem u_local : ∀ j, Utility.Local u j := by
  intro j x y h
  have he := h 1 (by simp [diagram])
  simp [u, he]

theorem collect_zero_of_member {vs : List (Valuation diagram.toFinBayesNet)}
  {v : Valuation diagram.toFinBayesNet} (hv : v ∈ vs) (x : diagram.Assignment)
  (hz : v.prob x = 0) : (collect vs).prob x = 0 := by
  induction vs with
  | nil => simp at hv
  | cons w vs ih =>
    change w.prob x * (collect vs).prob x = 0
    rcases List.mem_cons.1 hv with rfl | hv
    · rw [hz, zero_mul]
    · rw [ih hv, mul_zero]

/-- A genuinely zero OUTSIDE probability, without evidence and with a normalized closed model. -/
theorem zero_outside_context :
  (collect (outside (1 : diagram.V)
    (chanceStep (2 : diagram.V) (initial κ κ_local κ_nonnegative u u_local).valuations)))
    .prob
    (fun _ => true) = 0 := by
  let r := chanceValuation κ κ_local κ_nonnegative false
  have hi : r ∈ (initial κ κ_local κ_nonnegative u u_local).valuations := by
    apply List.mem_append_left
    exact List.mem_map.2 {false, Finset.mem_toList.2 (Finset.mem_univ _), rfl}
  have hc : r ∈ outside (2 : diagram.V) (initial κ κ_local κ_nonnegative u u_local).valuations :=
    List.mem_filter.2 {hi, by simp [r, chanceValuation, diagram]}
  have hd : r ∈ outside (1 : diagram.V)
    (chanceStep (2 : diagram.V) (initial κ κ_local κ_nonnegative u u_local).valuations) :=
    List.mem_filter.2 {List.mem_cons_of_mem _ hc, by simp [r, chanceValuation, diagram]}
  apply collect_zero_of_member hd
  simp [r, chanceValuation, κ]

theorem summed_child_retains_action_axis :
  (1 : diagram.V) ∈ (sumOut (2 : diagram.V) (chanceValuation κ κ_local κ_nonnegative true)).scope
  <- := by
  decide

theorem checked_zero_outside_model :
  solveGuarded κ κ_local κ_nonnegative u u_local order nf =
    some (solve κ κ_local κ_nonnegative u u_local order nf) :=
  solveGuarded_eq κ closed order nf κ_local κ_normalised κ_nonnegative u u_local

def zeroEvidence : Evidence Example.diagram where
  ancestors := ∅
  closed := fun _ h => False.elim (Finset.notMem_empty _ h)
  no_action := fun _ => Finset.notMem_empty _
  likelihood _ := 0
  localOn := fun _ _ => rfl
  nonneg := fun _ => le_refl 0

theorem guard_accepts_zero_evidence :
  checkedRun (Example.noForgetting.plan Example.order.no_self_info)
    (initialEvidence Example.kernels Example.local_kernels Example.nonnegative_kernels
      Example.utility Example.local_utility zeroEvidence) defaultStrategy =
  some (run (Example.noForgetting.plan Example.order.no_self_info)
    (initialEvidence Example.kernels Example.local_kernels Example.nonnegative_kernels
      Example.utility Example.local_utility zeroEvidence) defaultStrategy) :=
  checkedRun_generated_evidence_eq Example.kernels Example.closed Example.order Example.noForgetting
    Example.local_kernels Example.normalised_kernels Example.nonnegative_kernels
    Example.utility Example.local_utility zeroEvidence

```

```

theorem mass_check_rejects_zero_evidence :
  solveEvidenceGuarded Example.kernels Example.local_kernels Example.nonnegative_kernels
    Example.utility Example.local_utility Example.order Example.noForgetting zeroEvidence = none :=
  by
  rw [solveEvidenceGuarded_none_iff Example.kernels Example.closed Example.order Example.noForgetting
    Example.local_kernels Example.normalised_kernels Example.nonnegative_kernels
    Example.utility Example.local_utility zeroEvidence]
  simp [evidenceMass, zeroEvidence]

end
end InfluenceDiagramsProofs.DVE.GuardBoundary

```

Ordered local policies and a quantitative tie-gap contract

```

import InfluenceDiagramsProofs.Finite.Optimization
import BayesianNetworksProofs.Finite.NumericalContracts
import Mathlib.Tactic.Linarith

```

The local selector chooses the least maximizing state in a supplied finite linear order. For the raw-record model, states are $\text{Fin } n$ in checked state-position order. For another state representation, the caller must supply that order, not silently use alphabetical labels.

Only local action rows are maximized. The finite information table reconstructs the same admissible policy. A strict 2ε action gap protects the maximizing label against uniform score errors ε ; ties without a gap are intentionally not claimed stable.

```

set_option autoImplicit false

namespace InfluenceDiagramsProofs

open BayesianNetworksProofs BayesianNetworksProofs.FinBayesNet

noncomputable section

def maximizing {A : Type} [Fintype A] (f : A → ℝ) : Finset A := by
  classical
  exact Finset.univ.filter fun a => ∀ b, f b ≤ f a

theorem maximizing_nonempty {A : Type} [Fintype A] [Nonempty A] (f : A → ℝ) :
  (maximizing f).Nonempty := by
  classical
  obtain ⟨a, _, ha⟩ := Finset.exists_max_image Finset.univ f Finset.univ_nonempty
  exact ⟨a, Finset.mem_filter.2 ⟨Finset.mem_univ _, fun b => ha b (Finset.mem_univ _)⟩⟩

def firstArgmax {A : Type} [Fintype A] [Nonempty A] [LinearOrder A] (f : A → ℝ) : A :=
  (maximizing f).min' (maximizing_nonempty f)

theorem firstArgmax_maximizes {A : Type} [Fintype A] [Nonempty A] [LinearOrder A]
  (f : A → ℝ) (a : A) : f a ≤ f (firstArgmax f) := by
  have hm := Finset.min'_mem (maximizing f) (maximizing_nonempty f)
  exact (Finset.mem_filter.1 hm).2 a

/-- Exact first-state tie rule: no other maximizing state precedes the selected state. -/
theorem firstArgmax_first {A : Type} [Fintype A] [Nonempty A] [LinearOrder A]
  (f : A → ℝ) (a : A) (ha : ∀ b, f b ≤ f a) : firstArgmax f ≤ a :=
  Finset.min'_le _ a (Finset.mem_filter.2 ⟨Finset.mem_univ _, ha⟩)

theorem firstArgmax_stable {A : Type} [Fintype A] [Nonempty A] [LinearOrder A]
  (f g : A → ℝ) (ε : ℝ) (winner : A)
  (herr : ∀ a, |g a - f a| ≤ ε)
  (hgap : ∀ a, a ≠ winner → f a + 2 * ε < f winner) :
  firstArgmax g = winner := by
  by_contra he
  have hg := firstArgmax_maximizes g winner

```

```

have hhi := (abs_le.1 (herr (firstArgmax g))).2
have hlo := (abs_le.1 (herr winner)).1
have hstrict := hgap _ he
linarith

namespace FinInfluenceDiagram

variable {id : FinInfluenceDiagram} {d : id.D}
variable [LinearOrder (id.states (id.action d))]

def Policy.ofOrderedScore (score : id.Assignment → id.states (id.action d) → ℝ)
  (hscore : LocalOn (id.info d) score) : Policy id ℝ d :=
  Policy.ofFun (fun x ⇒ firstArgmax (score x))
    (fun x y h ⇒ congrArg firstArgmax (hscore x y h))

/-- A finite table on precisely the information variables, not on a fully observed history. -/
def orderedTable (score : id.Assignment → id.states (id.action d) → ℝ) :
  InfoAssignment id d → id.states (id.action d) :=
  fun i ⇒ firstArgmax (score (extendInfo d i))

theorem orderedTable_reconstruct (score : id.Assignment → id.states (id.action d) → ℝ)
  (hscore : LocalOn (id.info d) score) (x : id.Assignment) :
  orderedTable score (infoAssignment d x) = firstArgmax (score x) := by
  apply congrArg firstArgmax
  apply hscore
  intro v hv
  simp [extendInfo, infoAssignment, hv]

theorem orderedPolicy_table (score : id.Assignment → id.states (id.action d) → ℝ)
  (hscore : LocalOn (id.info d) score) (x : id.Assignment) (a : id.states (id.action d)) :
  (Policy.ofOrderedScore score hscore).kernel x a =
    if a = orderedTable score (infoAssignment d x) then 1 else 0 := by
  rw [orderedTable_reconstruct score hscore]
  rfl

theorem orderedPolicy_is_local (score : id.Assignment → id.states (id.action d) → ℝ)
  (hscore : LocalOn (id.info d) score) :
  LocalOn (id.info d) (Policy.ofOrderedScore score hscore).kernel :=
  (Policy.ofOrderedScore score hscore).localOn

theorem orderedPolicy_maximizes (score : id.Assignment → id.states (id.action d) → ℝ)
  (hscore : LocalOn (id.info d) score) (x : id.Assignment) (a : id.states (id.action d)) :
  score x a ≤ score x (orderedTable score (infoAssignment d x)) := by
  rw [orderedTable_reconstruct score hscore]
  exact firstArgmax_maximizes _ _

end FinInfluenceDiagram
end
end InfluenceDiagramsProofs

```

Roadmap and exact scope of the DVE theorem

```
import InfluenceDiagramsProofs.Finite.DVE.Evidence
```

The global optimum-existence result is now supplemented by an actual bucket solver: `DVE.solve_spec` proves its deterministic policies realize the reported value and attain the existing optimum. `NoForgettingOrder.plan` constructs its strong order from a complete perfect-recall decision list. No semantic legality trace is assumed. Probability independence is derived from the original finite diagram's topological order and kernel normalisation.

`DVE.solveEvidence_spec` and `DVE.solveEvidence_eq_optimal` extend this to likelihoods on an action-free chance-ancestral set, with explicitly positive evidence mass. `solveEvidenceChecked_eq` checks the driver-computed mass and rejects zero mass.

The formerly separate all-row diagnostic obligation is now closed: `all_guards_complete` and

`all_guards_complete_evidence` prove every bucket probability is action-independent on **every** row, including zero outside probability. The checked driver uses the source-style maximum of row maximum-minus-minimum at exact zero tolerance. `solveGuarded_eq` returns the same verified optimum; `solveEvidenceGuarded_none_iff` shows that the only remaining rejection for supported evidence is zero total evidence mass. No strict-positivity premise was added: positive normalized smoothing is only a proof device, and probability-expression continuity transfers the identities to zero.

This is **not** a byte-for-byte verification of Julia. Remaining refinements are:

- ordered arrays, state labels, reference resolution and the concrete min-fill scheduler;
- arbitrary finite local argmax ties versus Julia's first-label tie rule (the proved equivalence is realized optimal value, not identical policy tables or joint laws);
- explicit evidence conditioning versus a likelihood-factor representation, and zero-probability utility-table representatives (weighted valuations agree);
- Float64/tolerance behavior. Small action-dependent evidence probabilities can pass the source's tolerance guard and yield a wrong reported value; they are outside the theorem's action-independent evidence contract.

`Finite/OrderedPolicies.lean` now supplies an exact first-state local selector, an information table reconstruction theorem and a strict-gap numerical stability contract. This does not silently rewrite the existing DVE driver's selector or establish full first-label array identity. The BN proof dependency now includes concrete checked records/references, repeated slots, conditioned distributions, collect/distribute message correctness, moralized-ancestral d-separation soundness and conditional numerical error bounds.

The implementation-facing boundaries above must accompany any claim of Proposition 7. There are no unproved Lean declarations in this module.